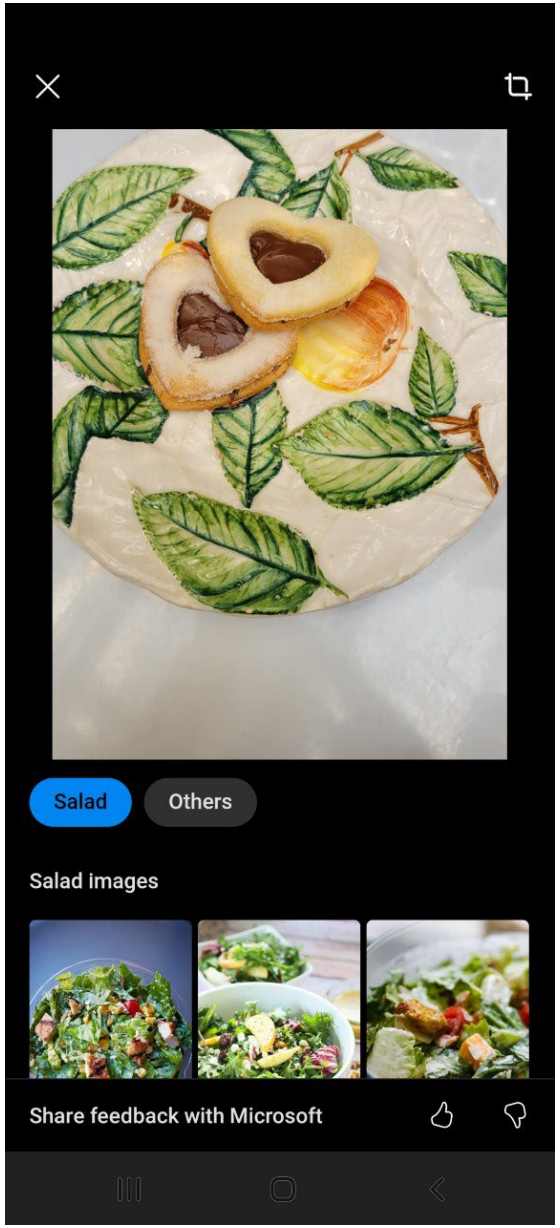
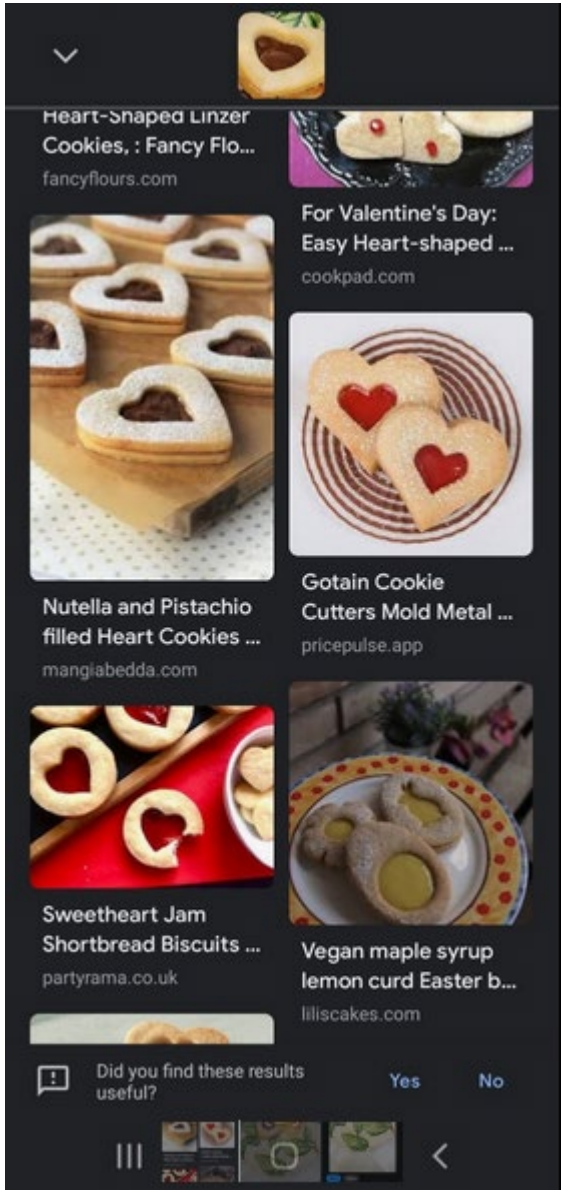
Convolutional Neural Networks


Salad
Others
Salad images
Share feedback with Microsoft


Google Lens


Heart-Shaped Linzer Cookies, : Fancy Flo...
fancyflours.com
For Valentine's Day: Easy Heart-shaped ...
cookpad.com
Nutella and Pistachio filled Heart Cookies ...
mangiabedda.com
Gotain Cookie Cutters Mold Metal ...
pricepulse.app
Sweetheart Jam Shortbread Biscuits ...
partyrama.co.uk
Vegan maple syrup lemon curd Easter b...
liliscakes.com
Did you find these results useful?
Yes
No

B / W Image 2x2px

Pixel 1	Pixel 2
Pixel 3	Pixel 4

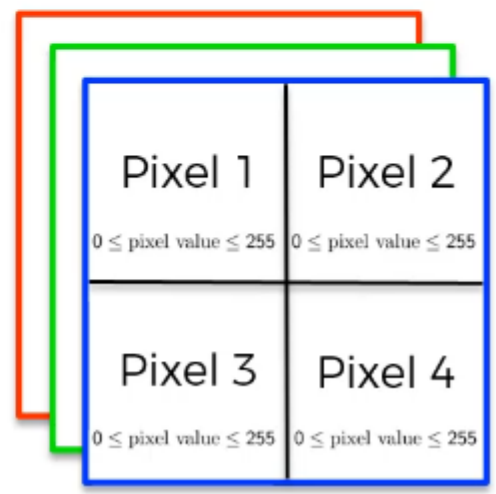
2d array

Pixel 1	Pixel 2
$0 \leq$ pixel value ≤ 255	$0 \leq$ pixel value ≤ 255
Pixel 3	Pixel 4
$0 \leq$ pixel value ≤ 255	$0 \leq$ pixel value ≤ 255

Colored Image 2x2px

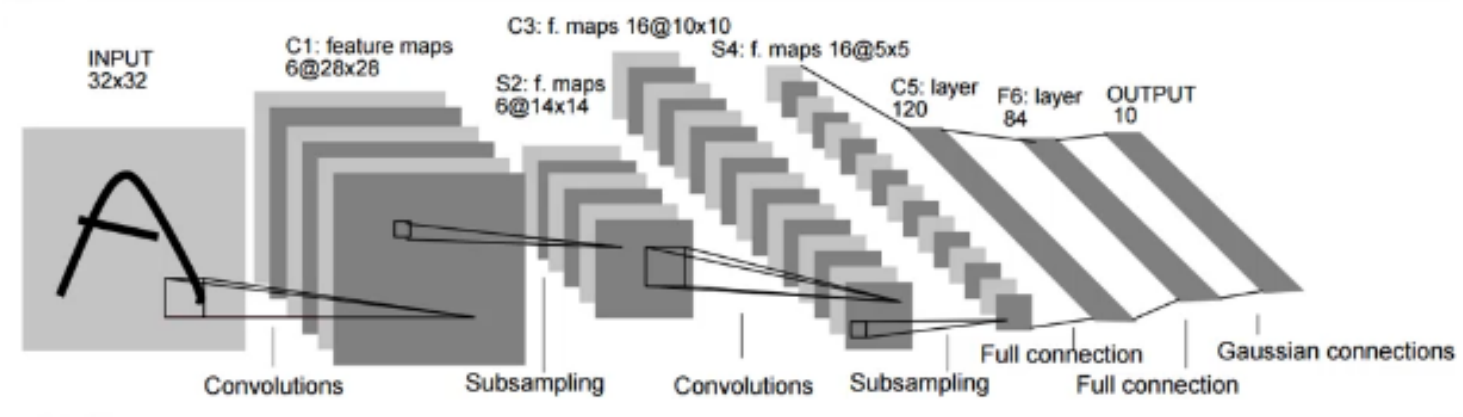
Pixel 1	Pixel 2
Pixel 3	Pixel 4

3d array



CNN Steps

- Convolution (with ReLU)
- Max Pooling
- Flattening
- Full Connection



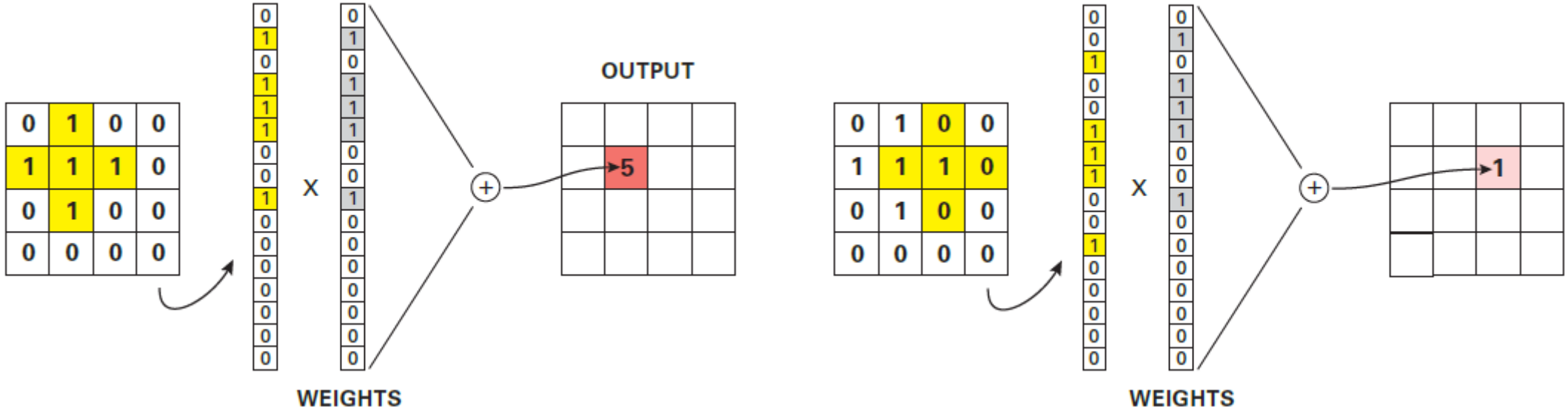
Why Convolution? To solve 3 main problems

1. Input images may have different size
   We would need to learn different W matrices for each size

2. Every pixel in input means that we need a LOT of weights in the network
   A generic input image is $x \in \mathbb{R}^{WHC}$ [Width, Height, Channel]
   First layer would have $W * H * C * D$ weights [D is the # of hidden units]

3. Patterns that occur in one location may not be recognized if occurred in a different location
   No **translation invariance** (weights are not shared across locations)

Why Convolution? To solve 3 main problems



- We can design a weight vector to act as a matched filter for detecting the desired cross-shape
- This will give a strong response of 5 if the object is on the left, but a weak response of 1 if the object is shifted over to the right

Step 1 - Convolution

- This is the transformation in mathematical terms

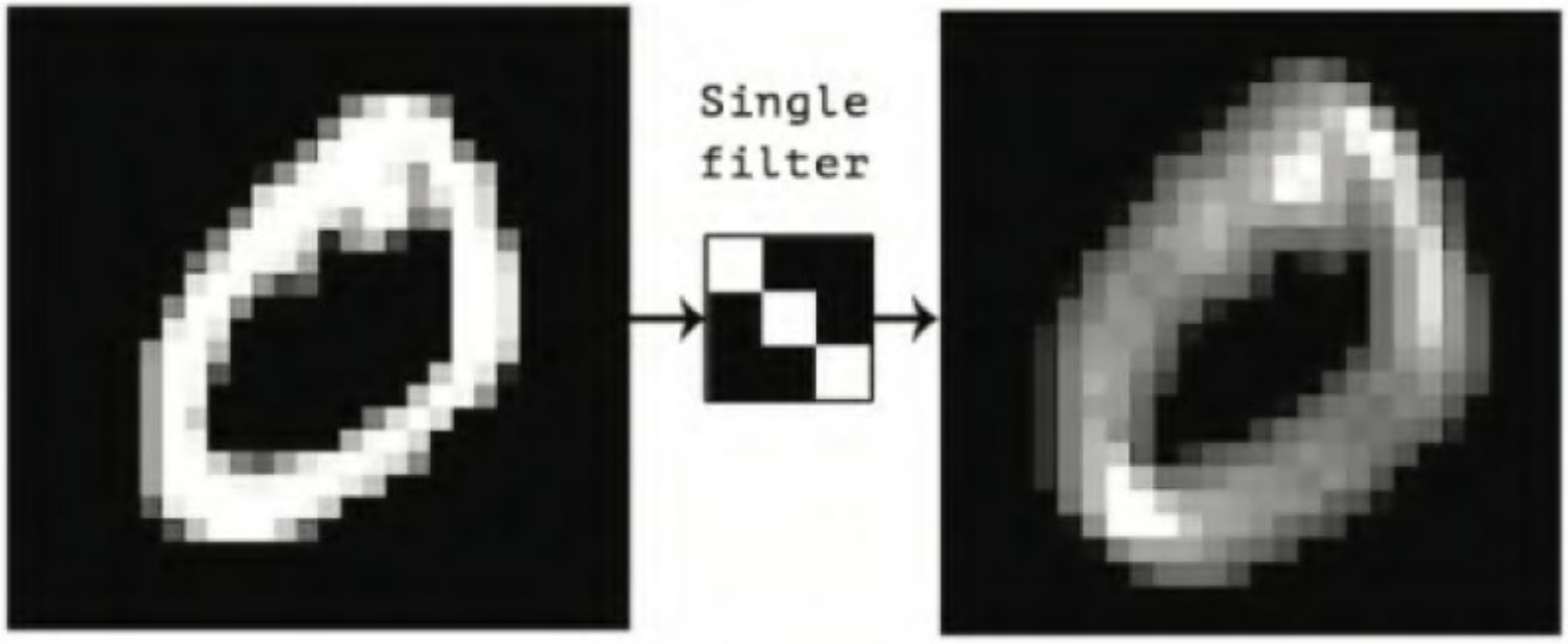
$$(f * g)(t) \stackrel{\text{def}}{=} \int_{-\infty}^{\infty} f(\tau)\, g(t-\tau)\, d\tau$$

- The verb to use is “to convolve”

  That translated from Math is ‘to combine’

Step 1 - Convolution

Example of a **Feature Detector** (aka **filter**)



Note that the bright spots occur where the filter pattern appears in the original picture

Step 1 - Convolution

Convolution for the **discrete case**

- Discrete convolution of $x = [1, 2, 3, 4]$ with $w = [5, 6, 7]$ to yield $z = [5, 16, 34, 52, 45, 28]$

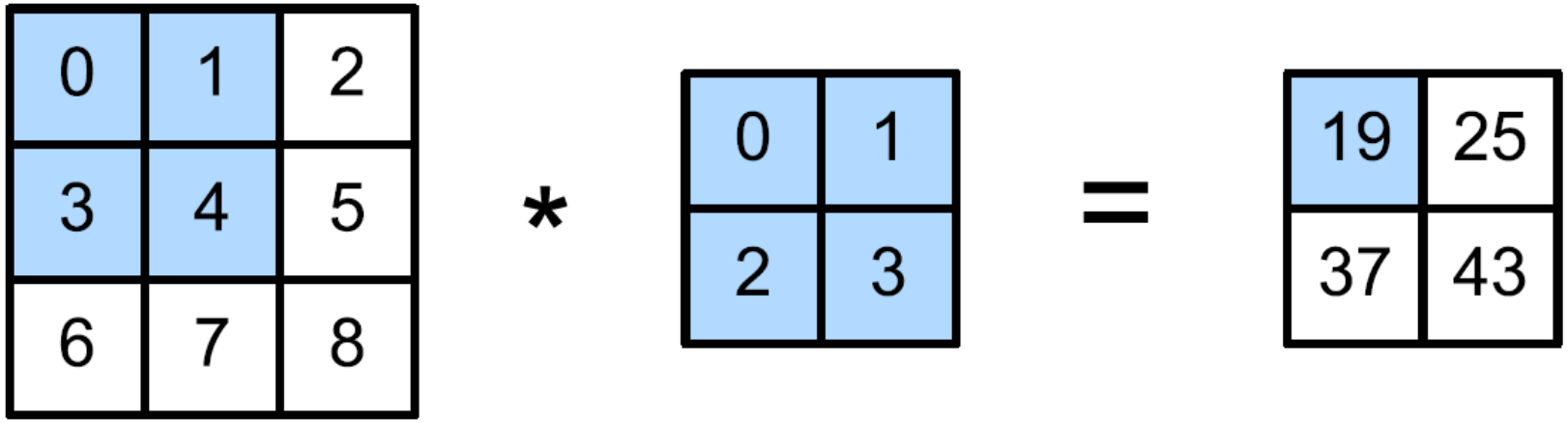
➢We see that this operation consists of “flipping” w and then “dragging” it over x, multiplying elementwise, and adding up the results.

-	-	1	2	3	4	-	-	
7	6	5	-	-	-	-	-	$z_0 = x_0 w_0 = 5$
-	7	6	5	-	-	-	-	$z_1 = x_0 w_1 + x_1 w_0 = 16$
-	-	7	6	5	-	-	-	$z_2 = x_0 w_2 + x_1 w_1 + x_2 w_0 = 34$
-	-	-	7	6	5	-	-	$z_3 = x_1 w_2 + x_2 w_1 + x_3 w_0 = 52$
-	-	-	-	7	6	5	-	$z_4 = x_2 w_2 + x_3 w_1 = 45$
-	-	-	-	-	7	6	5	$z_5 = x_3 w_2 = 28$

Step 1 - Convolution

Example of 1D Convolution

Example of 2D Convolution



Step 1 - Convolution

$$[\mathbf{W} \circledast \mathbf{X}](i,j) = \sum_{u=0}^{H-1} \sum_{v=0}^{W-1} w_{u,v} x_{i+u,j+v}$$

Example of 2D Convolution

0	1	2
3	4	5
6	7	8

*

0	1
2	3

=

19	25
37	43

Step 1 - Convolution

For example, convolving a (3 x 3) input X with a (2 x 2) filter to compute a (2 x 2) output Y:

$$\mathbf{Y} = \begin{pmatrix} w_1 & w_2 \\ w_3 & w_4 \end{pmatrix} \circledast \begin{pmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{pmatrix}$$

$$= \begin{pmatrix} (w_1 x_1 + w_2 x_2 + w_3 x_4 + w_4 x_5) & (w_1 x_2 + w_2 x_3 + w_3 x_5 + w_4 x_6) \\ (w_1 x_4 + w_2 x_5 + w_3 x_7 + w_4 x_8) & (w_1 x_5 + w_2 x_6 + w_3 x_8 + w_4 x_9) \end{pmatrix}$$

$$\mathbf{Y} = \begin{pmatrix} w_1 & w_2 \\ w_3 & w_4 \end{pmatrix} \circledast \begin{pmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{pmatrix}$$

$$= \begin{pmatrix} (w_1x_1 + w_2x_2 + w_3x_4 + w_4x_5) & (w_1x_2 + w_2x_3 + w_3x_5 + w_4x_6) \\ (w_1x_4 + w_2x_5 + w_3x_7 + w_4x_8) & (w_1x_5 + w_2x_6 + w_3x_8 + w_4x_9) \end{pmatrix}$$

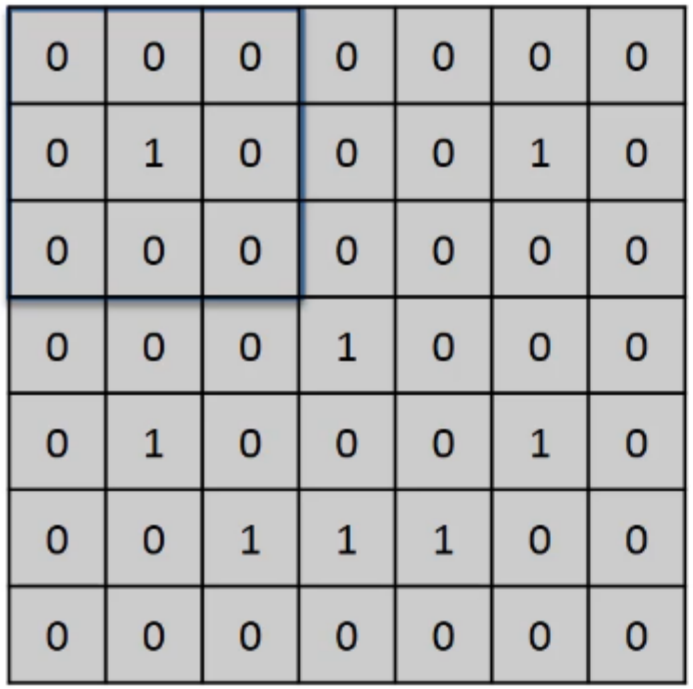
- Since convolution is a linear operator, we can represent it by matrix multiplication:

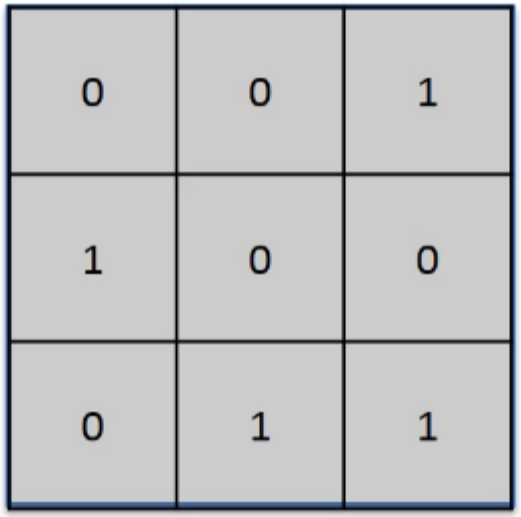
$$\left(\begin{array}{ccc|ccc|ccc} w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 \\ 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 \\ 0 & 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 \end{array}\right) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \end{pmatrix}$$

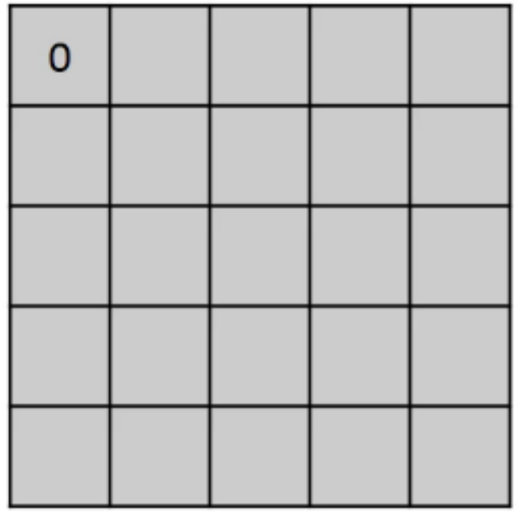
$$\left(\begin{array}{ccc|ccc|ccc} w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 & 0 \\ 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 & 0 & 0 \\ 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 & 0 \\ 0 & 0 & 0 & 0 & w_1 & w_2 & 0 & w_3 & w_4 \end{array}\right) \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \end{pmatrix} = \begin{pmatrix} w_1x_1 + w_2x_2 + w_3x_4 + w_4x_5 \\ w_1x_2 + w_2x_3 + w_3x_5 + w_4x_6 \\ w_1x_4 + w_2x_5 + w_3x_7 + w_4x_8 \\ w_1x_5 + w_2x_6 + w_3x_8 + w_4x_9 \end{pmatrix}$$

- CNNs are similar to MLPs, but their weight matrices have a special sparse structure

  Elements are tied across spatial locations, achieving translation invariance and significantly reducing the number of parameters compared to standard fully connected layers in MLPs.







We **multiply** together the elements of these 3x3 matrices, position by position.
Then, we **sum** all the results together

All the multiplications are 0x1 or 1x0, so the sum is equal to 0

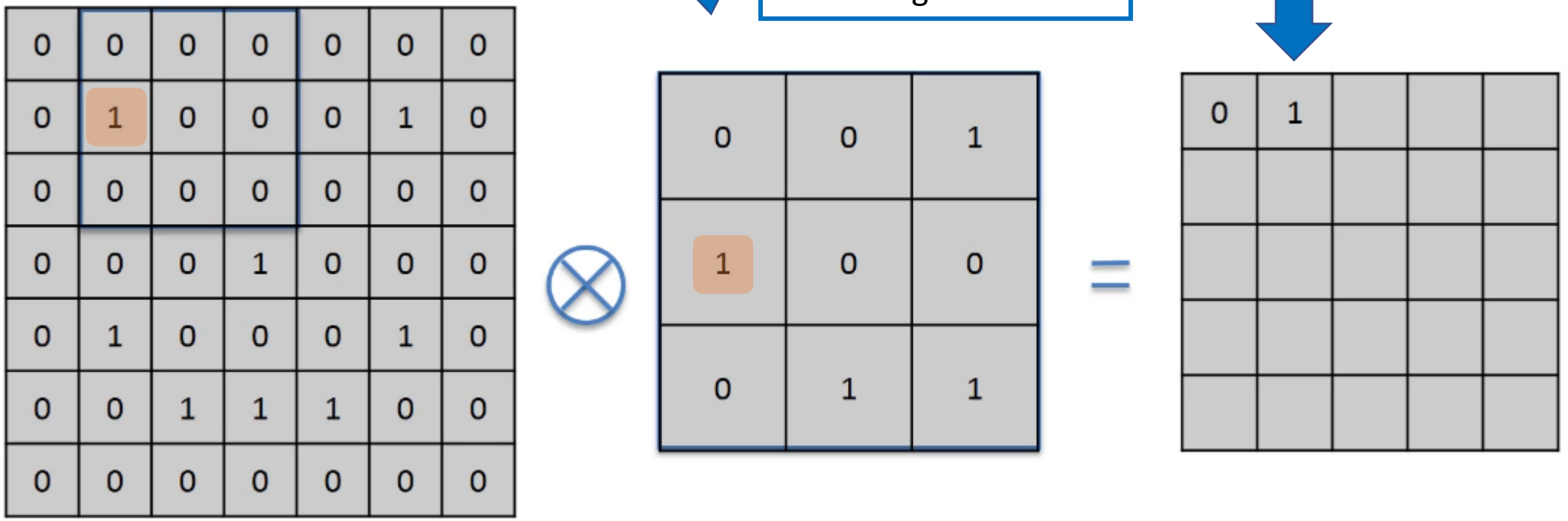
Input Image
Simplified as just 1s and 0s per pixel

Feature Detector
AKA Filter
Usually, 3x3 but it could be larger
It can contain negative values too

Feature Map
Simplified as just 1s and 0s per pixel

We **multiply** together the elements of these 3x3 matrices, position by position.
Then, we **sum** all the results together

Now, one multiplication is 1x1



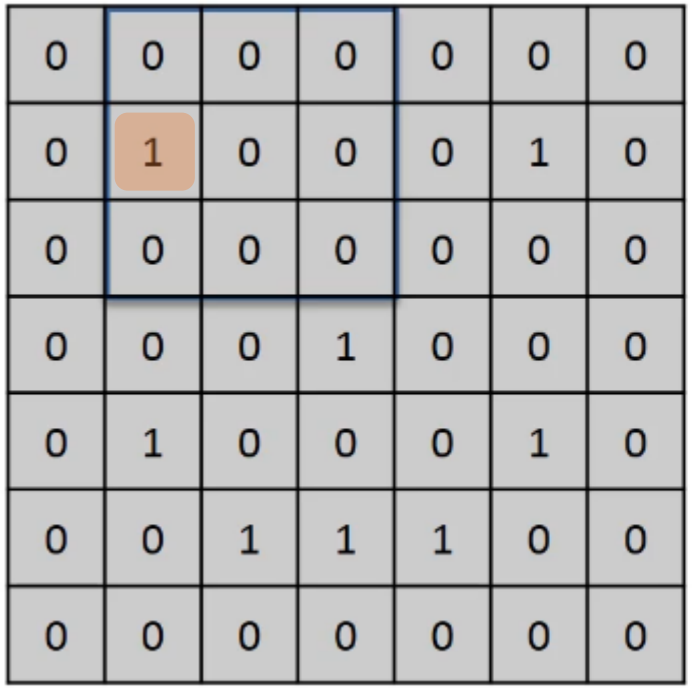
Input Image
Simplified as just 1s and 0s per pixel

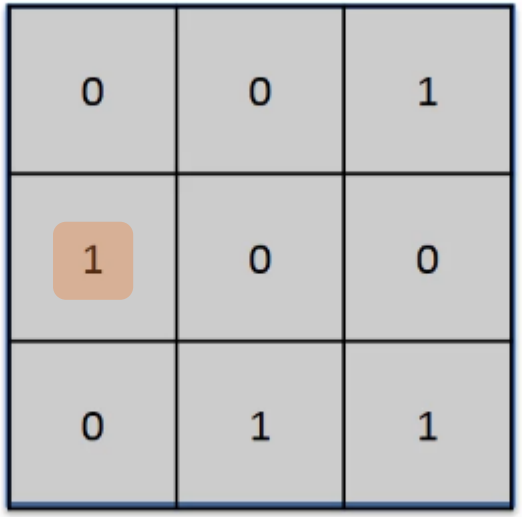
Feature Detector
AKA Filter
Usually, 3x3 but it could be larger
It can contain negative values too

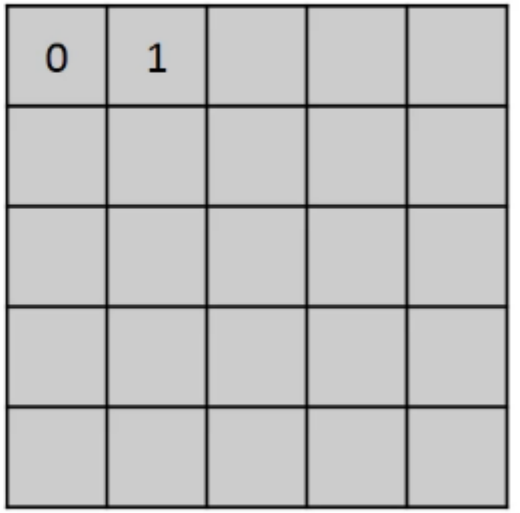
Feature Map
Simplified as just 1s and 0s per pixel

The movement of this matrix is called **stride**

- Here we have a stride of 1 pixel







Input Image

Simplified as just 1s and 0s per pixel

Feature Detector

AKA Filter

Usually, 3x3 but it could be larger

It can contain negative values too

Feature Map

Simplified as just 1s and 0s per pixel

After 10 strides

- Best values for the stride is by 1 or 2 pixels

0	0	0	0	0	0	0
0	1	0	0	0	1	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

0	0	1
1	0	0
0	1	1

0	1	0	0	0
0	1	1	1	

Input Image

Simplified as just 1s and 0s per pixel

Feature Detector

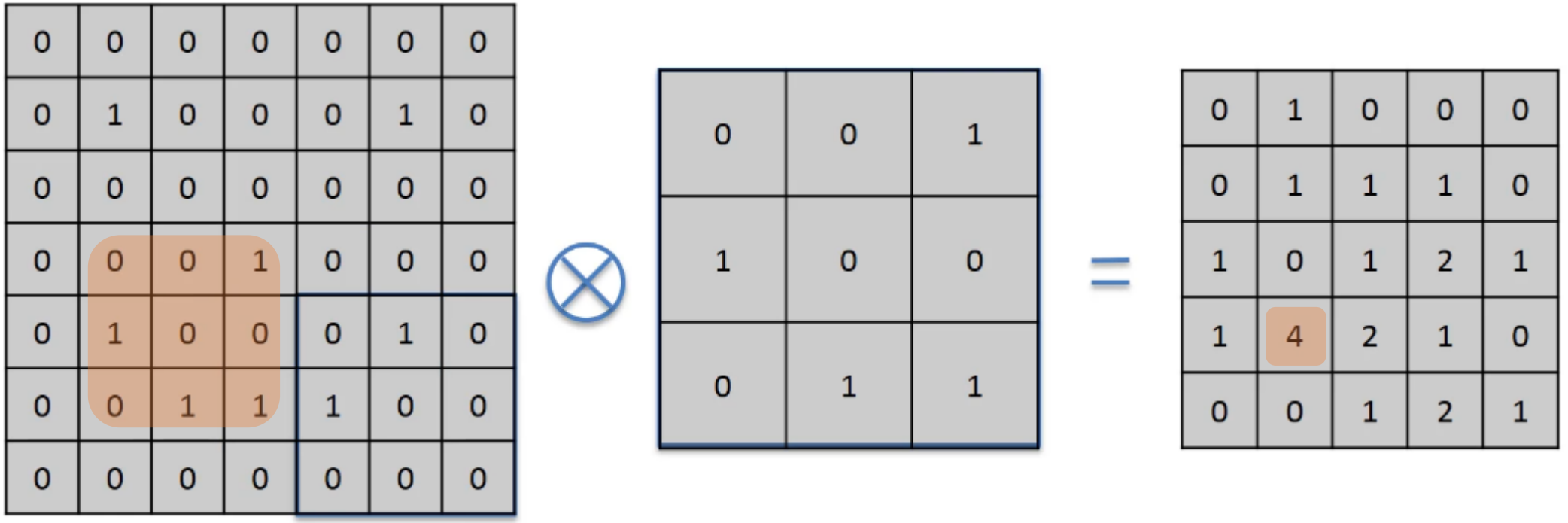
AKA Filter

Usually, 3x3 but it could be larger

It can contain negative values too

Feature Map

Simplified as just 1s and 0s per pixel



Input Image

Simplified as just 1s and 0s per pixel

Feature Detector

AKA Filter

Usually, 3x3 but it could be larger

It can contain negative values too

Feature Map

Simplified as just 1s and 0s per pixel

Step 1 - Convolution

- What is the **Feature Map**? What did we obtain?

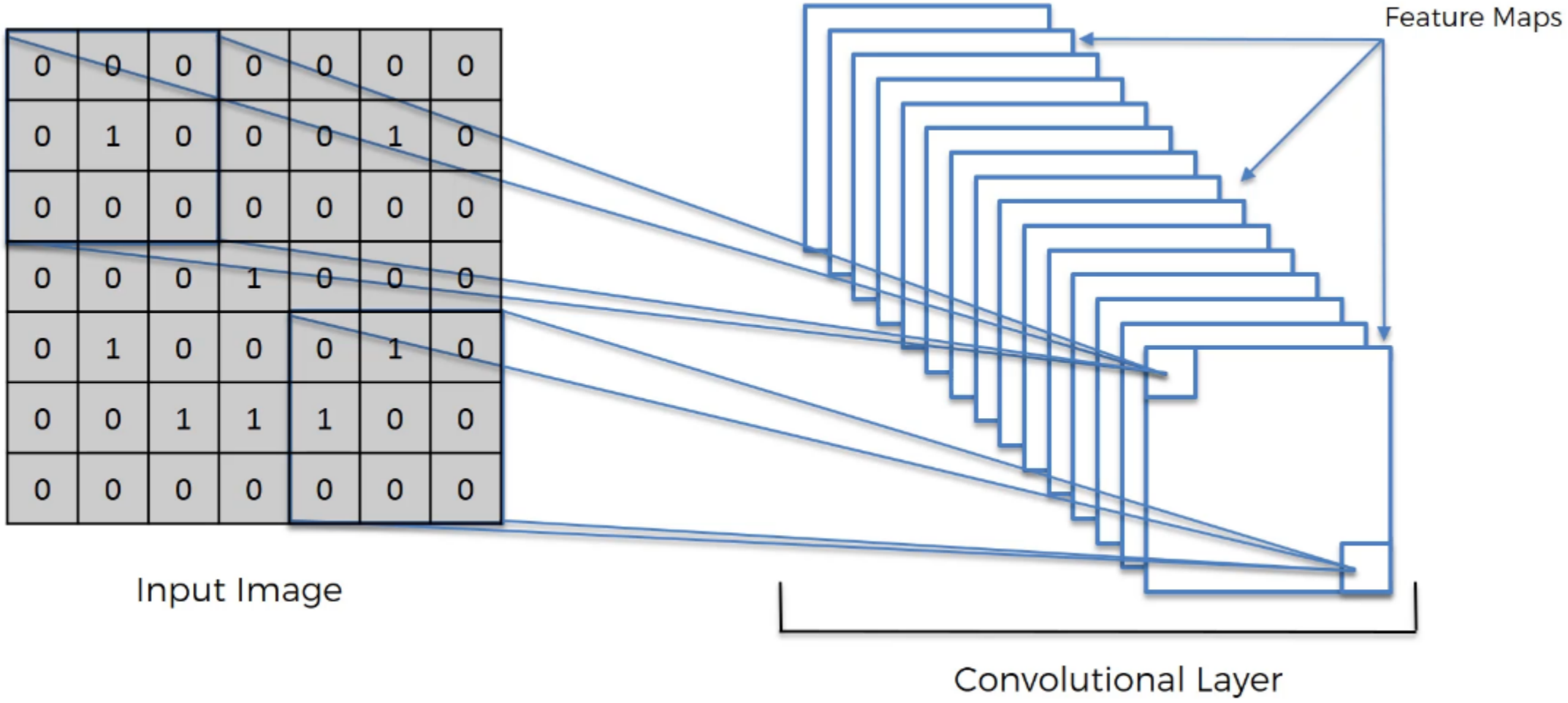
0	1	0	0	0
0	1	1	1	0
1	0	1	2	1
1	4	2	1	0
0	0	1	2	1

1. We reduced the size of the original matrix
   The more the larger is the **stride**

2. The higher are the numbers, the more precisely the detected feature appears in the original image

Note that it still preserves the spatial relationships between pixels

- We don't lose the pattern



- The network will be trained to recognize a series of Features that are then stored in a collection of Feature Maps

  This is the **Convolutional Layer**

Step 1 - Convolution

- Can I have convolution that mantains the size of the original image?

- Yes, it is called **Same Convolution**, and it uses **zero-padding**

  It basically adds a border of 0s to the image

How big is the Feature Map?

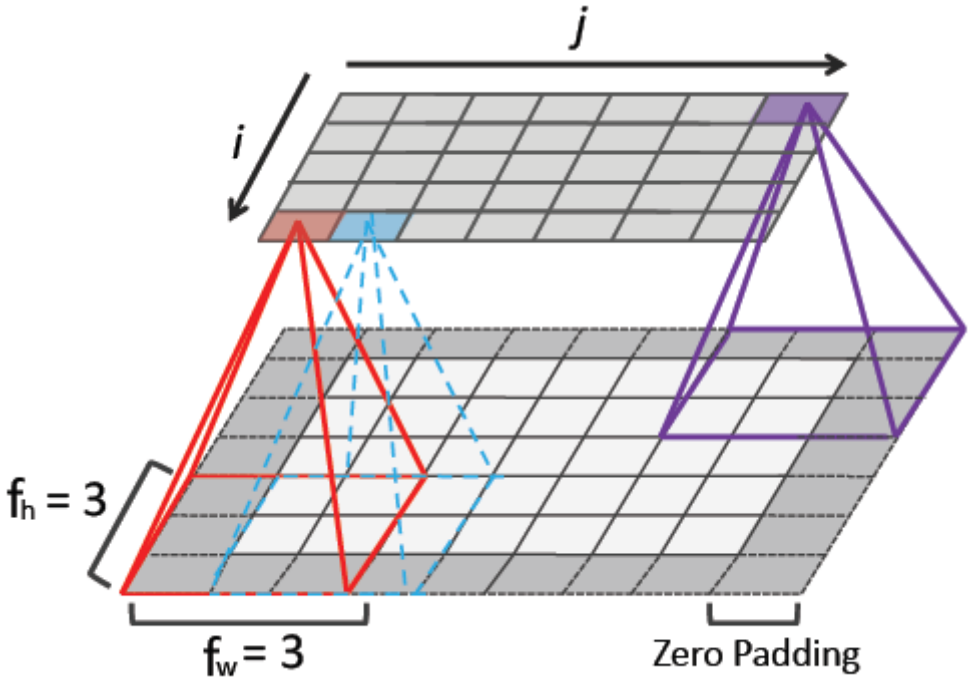
- If the input has size $x_h * x_w$, we use a filter of size $f_h * f_w$, we use zero padding on each side of size p_h and p_w, then the output has the following size:

$$(x_h + 2p_h - f_h + 1) * (x_w + 2p_w - f_w + 1)$$

  - For example, if $p = 1$, $f = 3$, $x_h = 5$ and $x_w = 7$, the output has size:

$$(5 + 2 - 3 + 1) * (7 + 2 - 3 + 1) = 5 * 7$$

> ➢ If we set $2p = f - 1$, then the output will have the same size as the input



$$(x_h + 2p_h - f_h + 1) * (x_w + 2p_w - f_w + 1)$$

$p = 1$
$f = 3$
$x_h = 5$
$x_w = 7$

$$(5 + 2 - 3 + 1) * (7 + 2 - 3 + 1) = 5 * 7$$

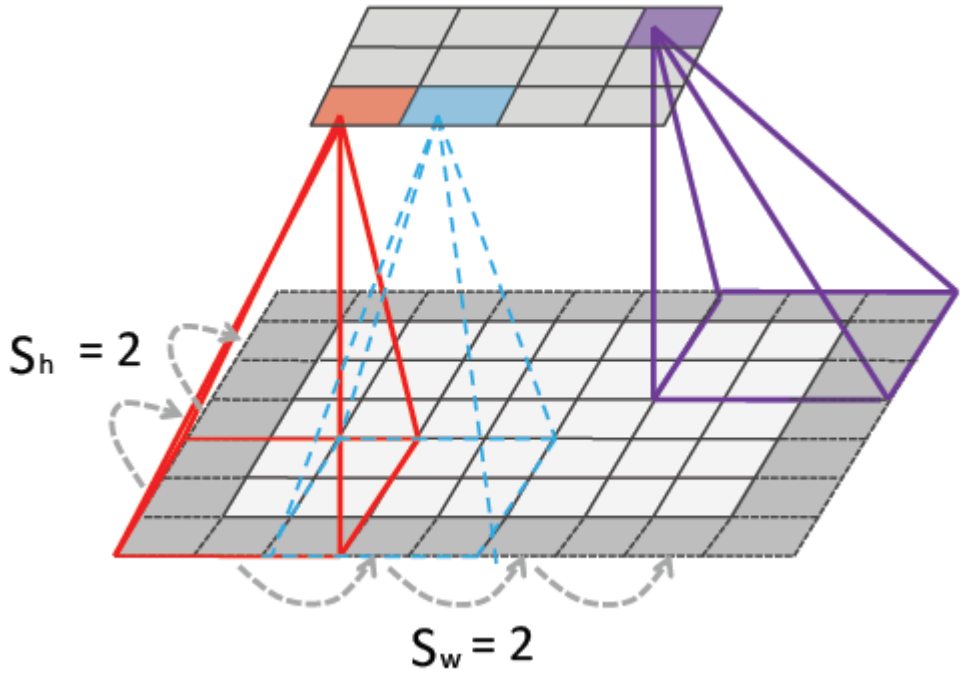
How stride affects the size of the Feature Map?

- If the input has size $x_h * x_w$, we use a filter of size $f_h * f_w$, we use zero padding on each side of size p_h and p_w, and we use strides of size s_h and s_w, then the output has the following size:

$$\frac{(x_h + 2p_h - f_h + s_h)}{s_h} * \frac{x_w + 2p_w - f_w + s_w}{s_w}$$

  - For example, if $p = 1$, $f = 3$, $x_h = 5$, $x_w = 7$, $s_h = 2$ and $s_w = 2$, the output has size:

$$\frac{(5 + 2 - 3 + 2)}{2} * \frac{(7 + 2 - 3 + 2)}{2} = 3 * 4$$



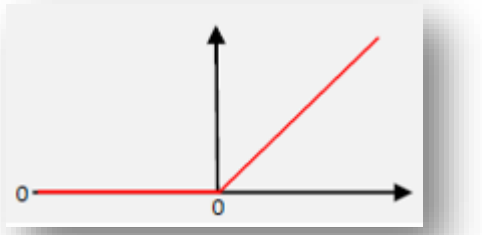
$$\frac{(x_h + 2p_h - f_h + s_h)}{s_h} * \frac{x_w + 2p_w - f_w + s_w}{s_w}$$

$p = 1$
$f = 3$
$x_h = 5$
$x_w = 7$
$s_h = 2$
$s_w = 2$

$$\frac{(5 + 2 - 3 + 2)}{2} * \frac{(7 + 2 - 3 + 2)}{2} = 3 * 4$$

Step 1 - Convolution

- After creating the set of Feature Maps, we need to add some non-linearity
- We apply the **Rectifier** activation function (**ReLU**)



➢All the negative values are filtered out

- This helps reducing the graduality of the color palette.

  Example: In a picture, colors go from bright to dark gradually.

  If we remove the negatives, it will change more abruptly. Less linearly!

Step 2 - Pooling

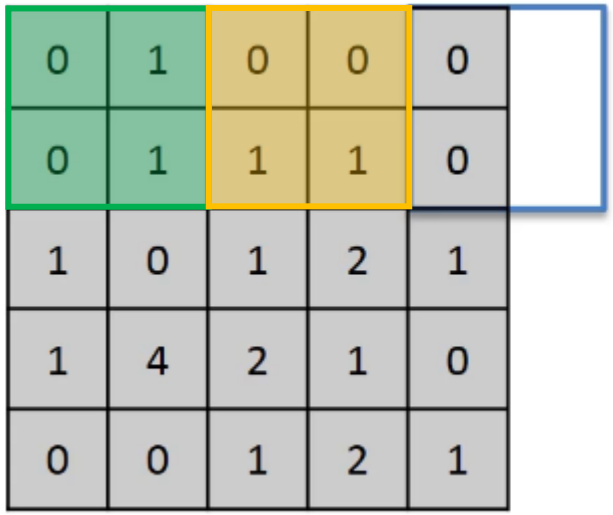
- Convolution, though, has equivariance

  It preserves info about the location of the patterns

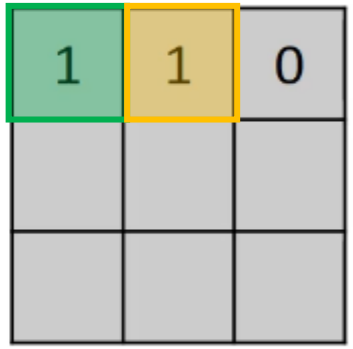
- We need to ensure that our neural network has a property called **translation invariance**

  Basically, that it doesn't care where the features are and doesn't care if the features are a bit tilted, different, relatively closer or farer apart, and so on.

Step 2 - Pooling



Max Pooling



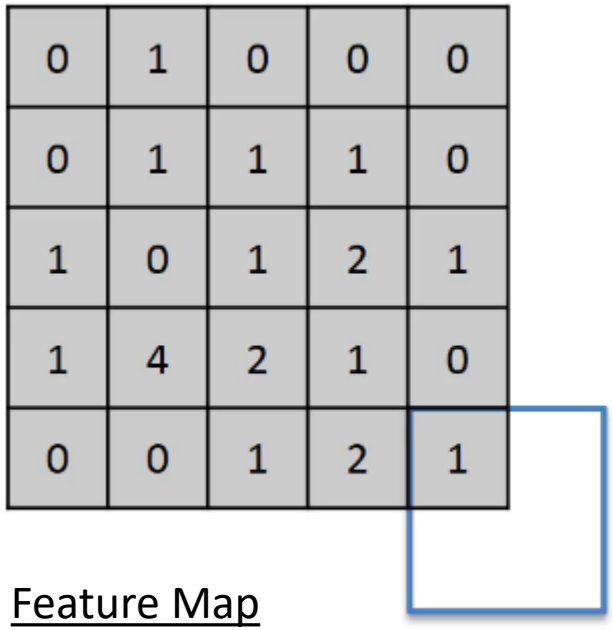
Feature Map

- We pick **the maximum value** in a 2x2 sub-square (the size can vary)
- Stride is 2 here, but it can vary
- If at the end of the row, we just continue

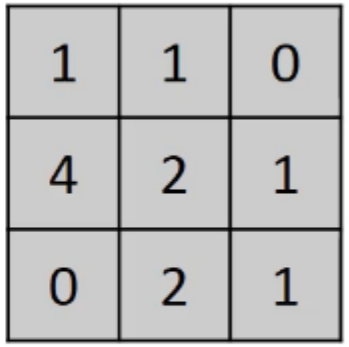
Pooled Feature Map

- Each position contains **the maximum value** found in the sub-squares

Step 2 - Pooling



Max Pooling



Feature Map

- We pick **the maximum value** in a 2x2 sub-square (the size can vary)
- Stride is 2 here, but it can vary
- If at the end of the row, we just continue

Pooled Feature Map

- Each position contains **the maximum value** found in the sub-squares

Step 2 - Pooling

Feature Map

0	1	0	0	0
0	1	1	1	0
1	0	1	2	1
1	4	2	1	0
0	0	1	2	1

Pooled Feature Map

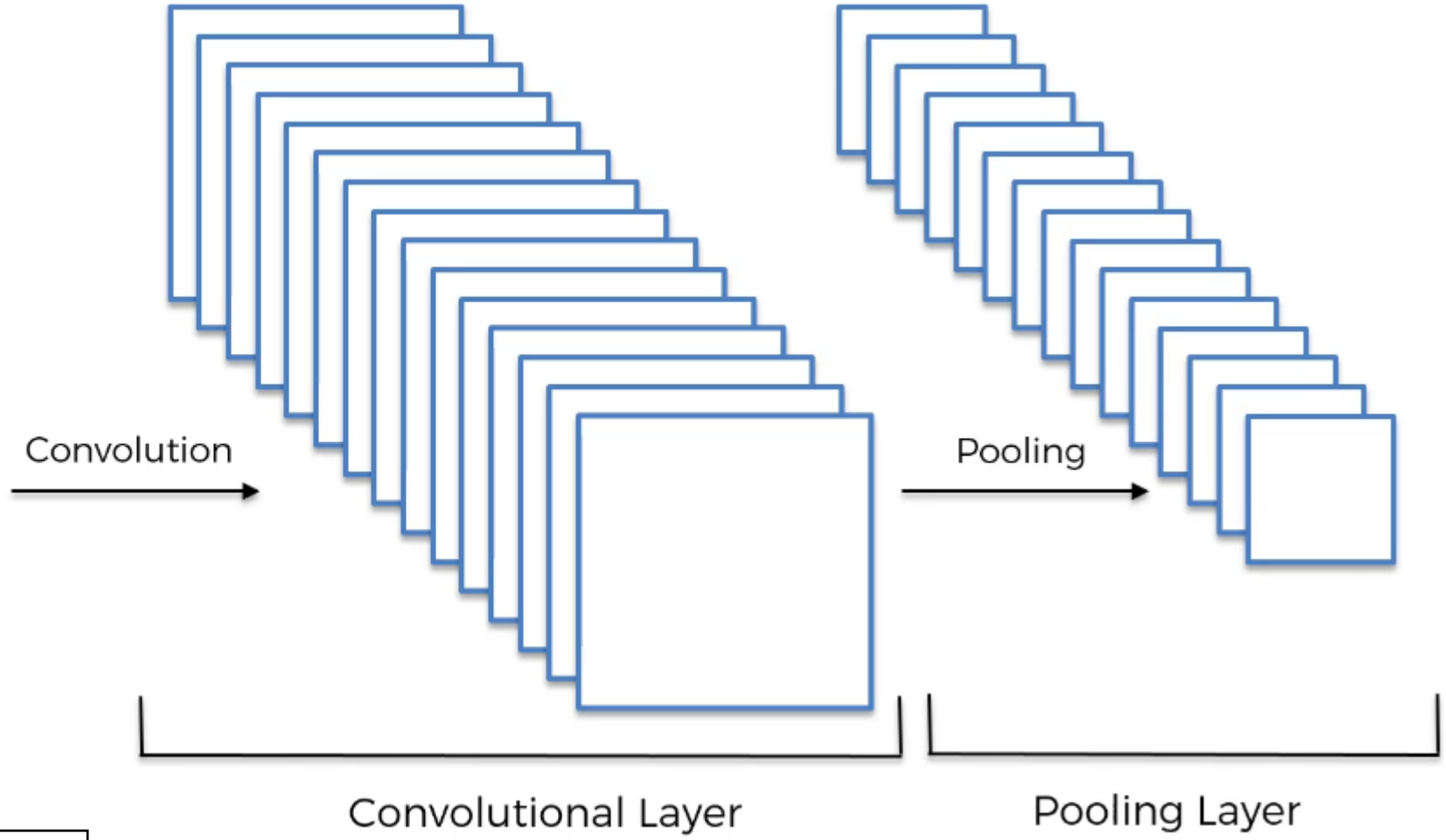
1	1	0
4	2	1
0	2	1

- In the **Pooled Feature Map**, we pick the maximum value in a small area (the sub-square)
  If in the Feature Map, the maximum value is moved in a different position, it will still be recognized and passed to the Pooled Feature Map
- We are also reducing the size of the Map, helping the work of the final layer
- Also, it reduces overfitting, because we are not training on the actual training data anymore

Convolution + Pooling

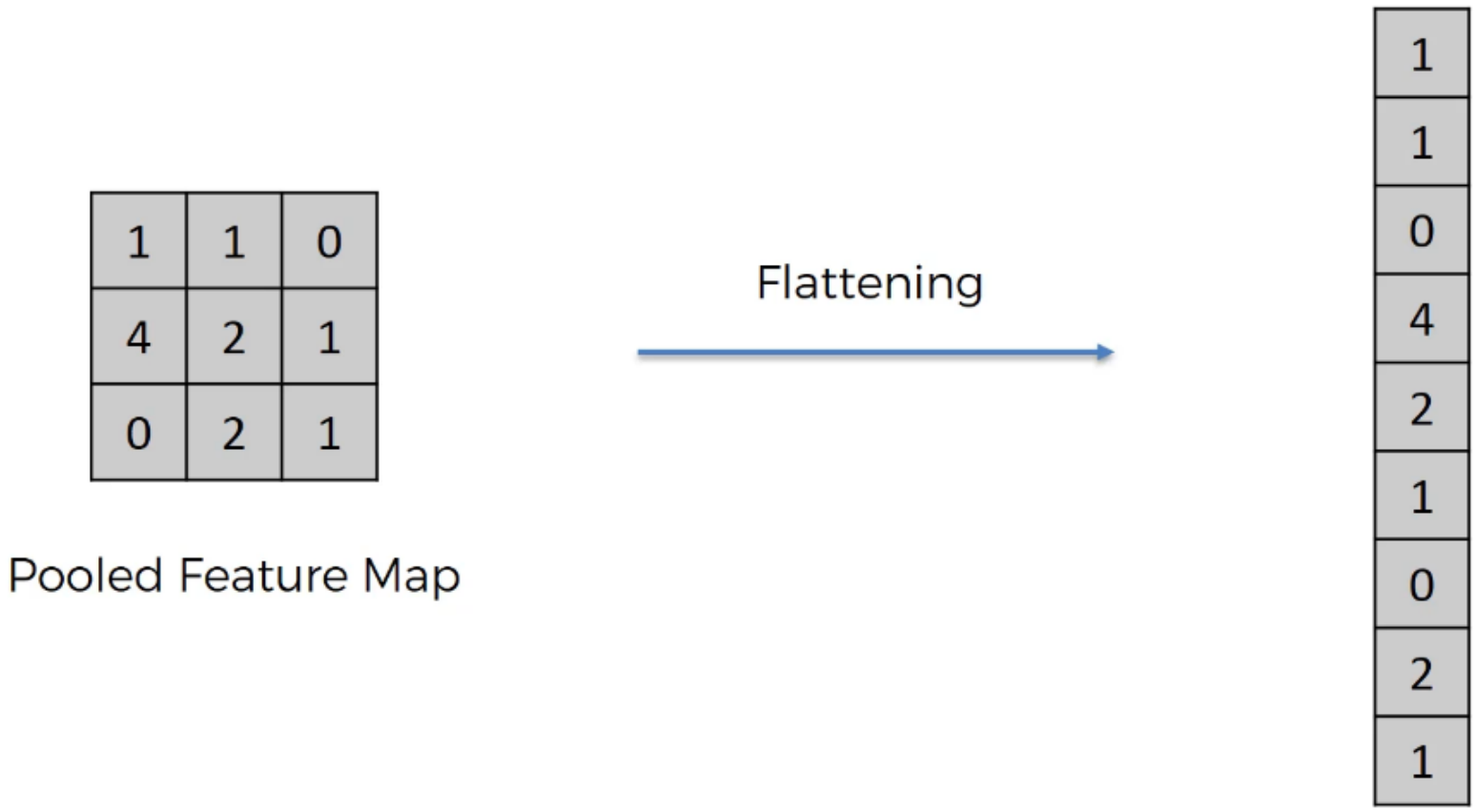
0	0	0	0	0	0	0
0	1	0	0	0	1	0
0	0	0	0	0	0	0
0	0	0	1	0	0	0
0	1	0	0	0	1	0
0	0	1	1	1	0	0
0	0	0	0	0	0	0

Input Image



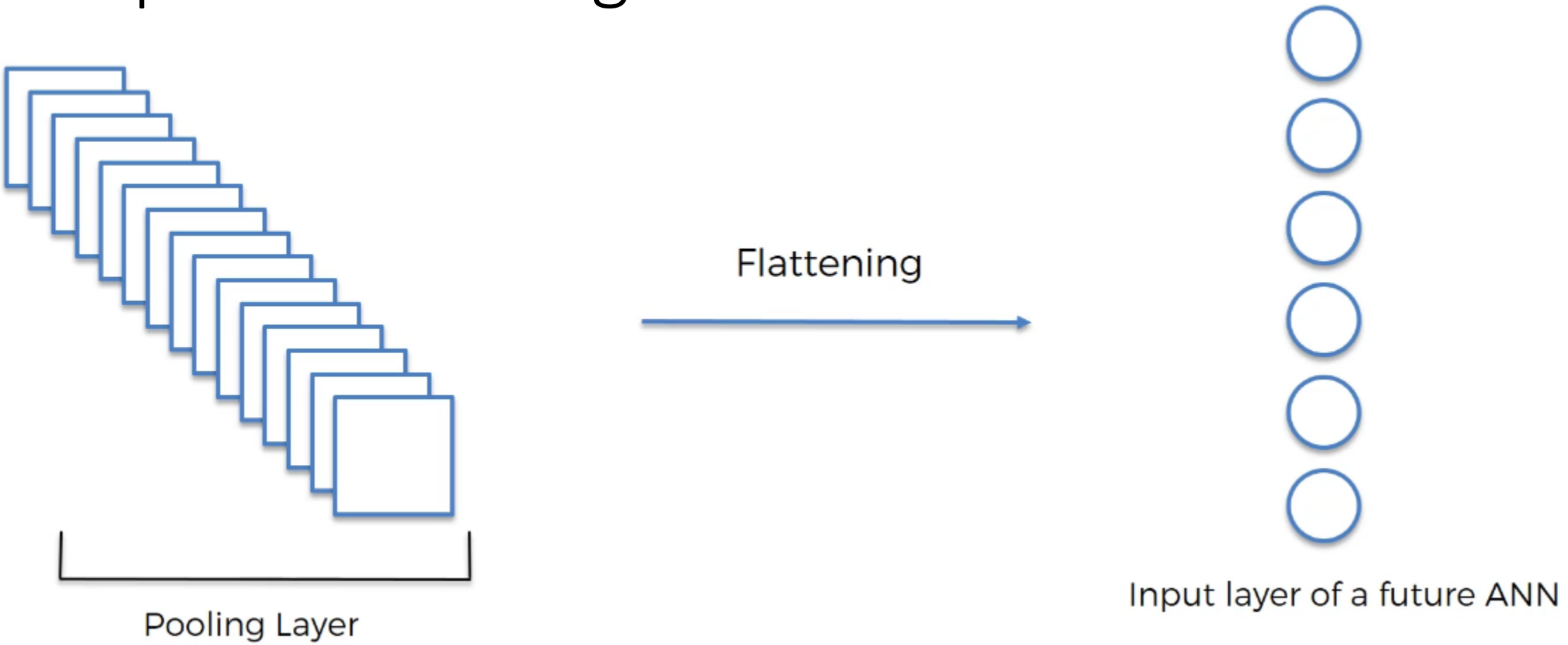
Note: Pooling is applied to any Convolutional Layer

Step 3 - Flattening



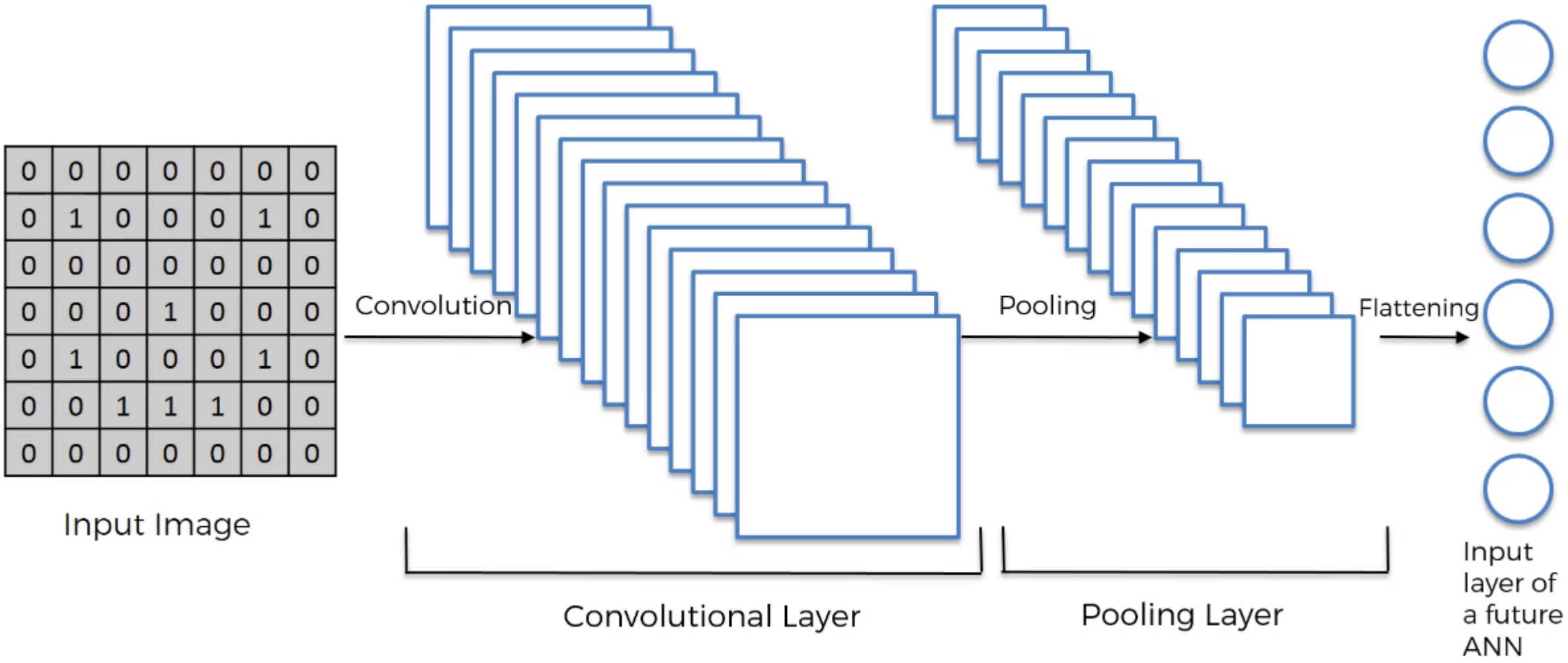
- Simply, we convert the matrices in a vector (by rows)

Step 3 - Flattening



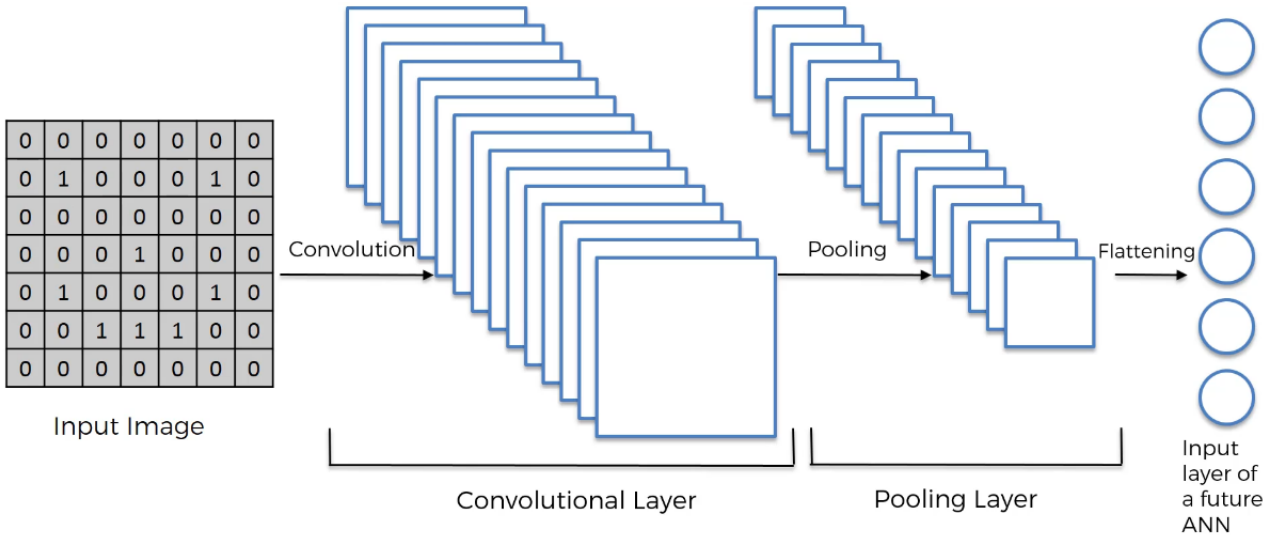
- Simply, we convert the matrices in a vector (by rows)

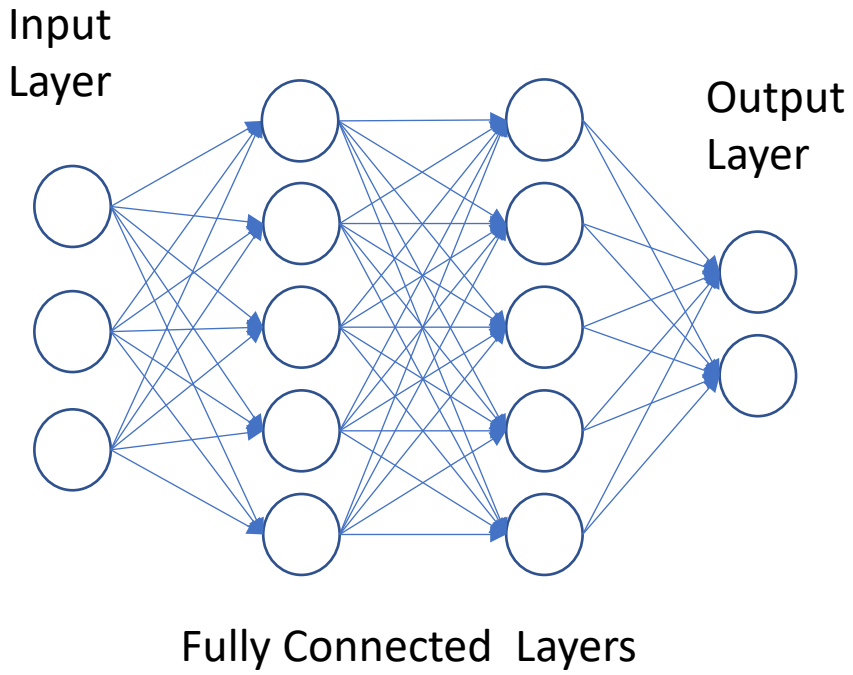
Convolution + Pooling + Flattening



Step 4 – Full Connection

- We finally attach the ANN



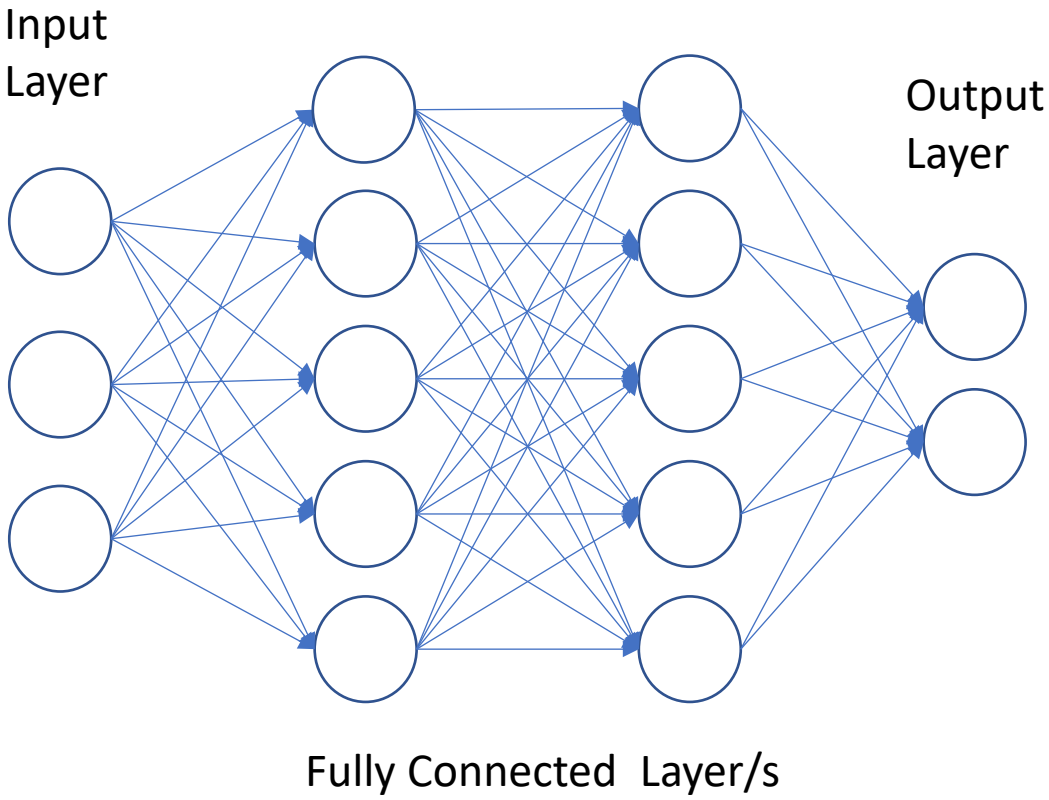


Note that in CNNs, the hidden layers are called "Fully Connected Layers"

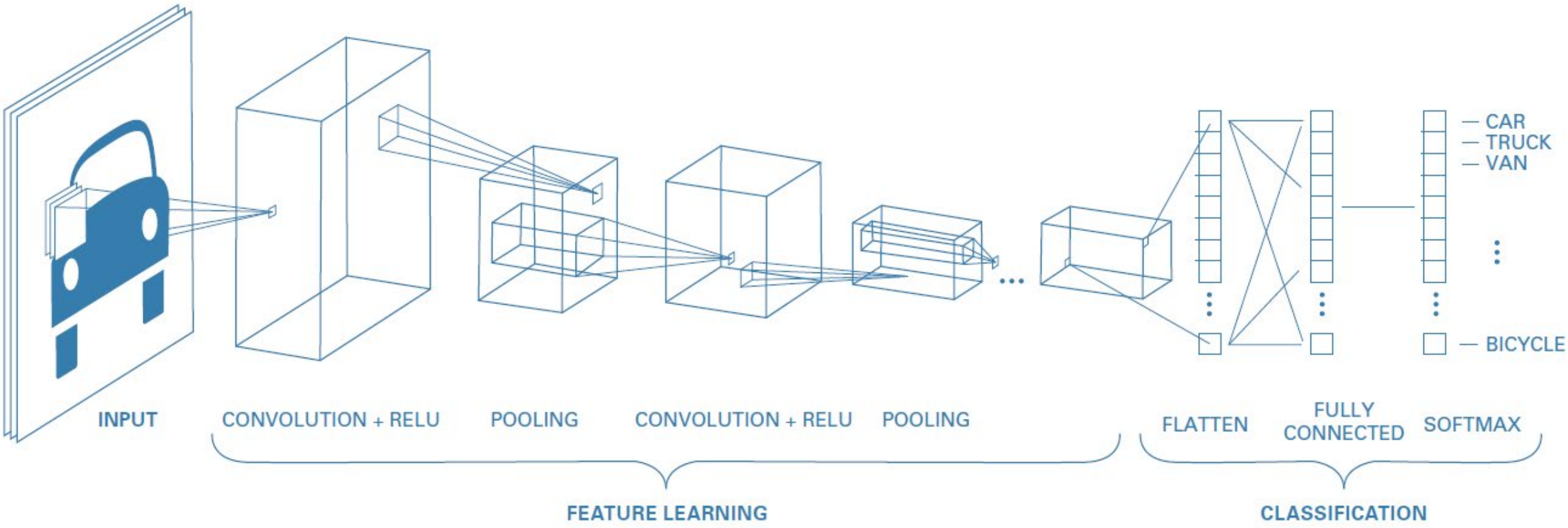
Step 4 – Full Connection

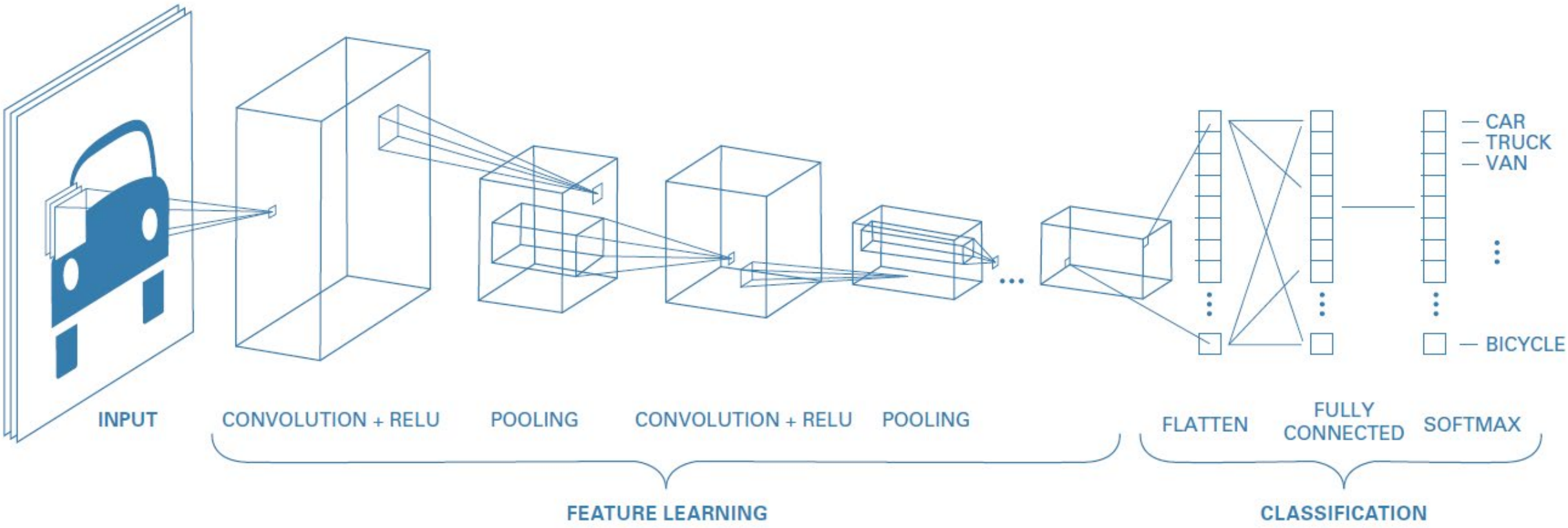
- In this network, though, the backpropagation will not simply identify the weights to update, but also the "features"

- In fact, the Feature Detectors are also modified based on the error (via gradient descent)



Step 4 – Full Connection





- What if we add more Convolutional and Pooling layers? The network will become very deep…

➢ Vanishing or exploding gradient problem!

Vanishing or Exploding gradient

- A common solution to this problem is to add extra layers to the model
  - Layers that reduce overfitting!

- We want to standardize the statistics of the hidden units
  - Zero mean and unit variance
  - This is similar to input standardization

- Alternatively, we can modify the Loss function to penalize greater weight values

Batch Normalization

- The most popular normalization layer is called **batch normalization**

  Distribution of the activations within a layer has zero mean and unit variance, when averaged across the samples in a minibatch

  We replace the activation vector z_n (input to a given layer) for example with $\tilde{z}_n$

Batch Normalization

$$\tilde{z}_n = \gamma \odot \hat{z}_n + \beta$$

$$\hat{z}_n = \frac{z_n - \mu_B}{\sqrt{(\sigma_B^2 + \epsilon)}}$$

$\boldsymbol{B}$ is the minibatch containing the example n
$\boldsymbol{\mu_B}$ is the mean of the activations in this batch
$\boldsymbol{\sigma_B^2}$ is the corresponding variance
$\hat{\boldsymbol{z}}_{\boldsymbol{n}}$ is the standardized activation vector
$\tilde{\boldsymbol{z}}_{\boldsymbol{n}}$ is the shifted and scaled version (output of the BN layer)
$\boldsymbol{\epsilon} > 0$ is a small constant
$\boldsymbol{\beta}$ (shift) and $\boldsymbol{\gamma}$ (scale) are learnable parameters

$$\mu_B = \frac{1}{|B|}\sum_{z \in B} z$$

$$\sigma_B^2 = \frac{1}{|B|}\sum_{z \in B}(z - \mu_B)^2$$

$\odot$ is elementwise product

Batch Normalization – **Test set**

- Since this transformation is differentiable, we can easily pass gradients back to the input of the layer and to the BN parameters γ and β

➢ And during test? We have a single input, so we cannot compute batch statistics. The standard solution to this is as follows:

1. After training, compute μ_l and σ_l^2 for layer l across all the examples in the training set (the full training set) [of course, a moving average]
2. Then, save these parameters, and add them to the list of other parameters for the layer (β_l and γ_l)
3. At test time, we use these saved training values instead of computing statistics from the test batch

Note that, when using a model with BN, we need to specify if we are using it for inference (test) or training

Batch Normalization – Last words

- The benefits of batch normalization (in terms of training speed and stability) are great!

  Especially for deep CNNs

The exact reasons for this are still unclear

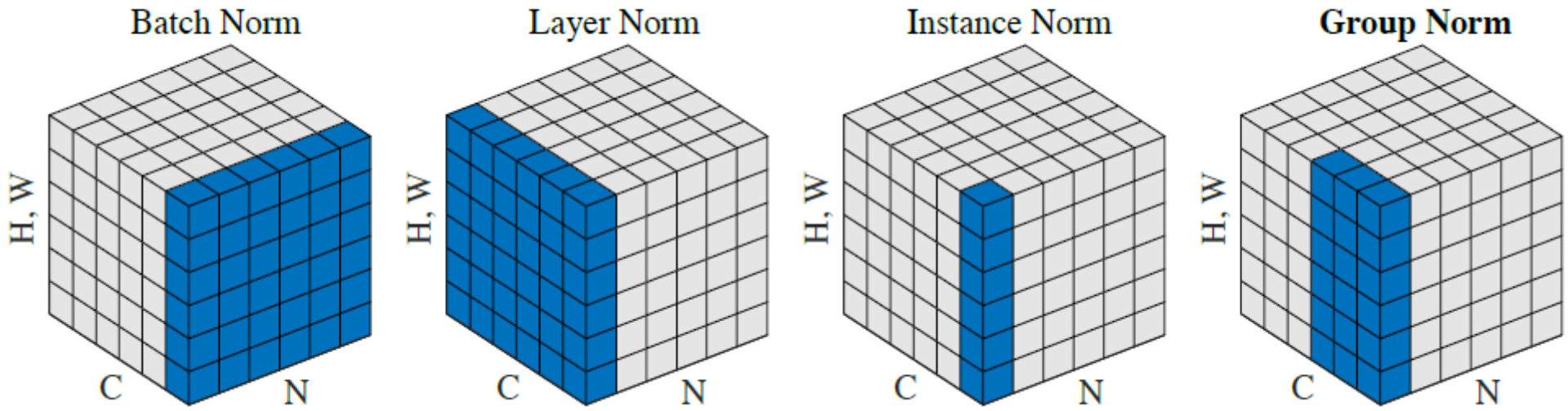
- BN seems to make the optimization landscape significantly smoother
- It also reduces the sensitivity to the learning rate

However, it struggles when the batch size is small

The estimated mean and variance parameters can be unreliable

- One solution is to compute the mean and variance by pooling statistics across other dimensions of the tensor, but not across examples in the batch

Other types of Normalization



- The **blue pixels** within the feature map are **normalized** using the same mean and variance. These statistics are computed by aggregating the values of these pixels.

  - **Batch Normalization (batch norm)**: Applies normalization across the entire batch.
  - **Layer Normalization (layer norm)**: Normalizes within each channel independently.
  - **Instance Normalization (instance norm)**: Normalizes per instance (sample) independently.
  - **Group Normalization (group norm)**: Divides channels into groups (here, 2 groups of 3 channels) and normalizes within each group.

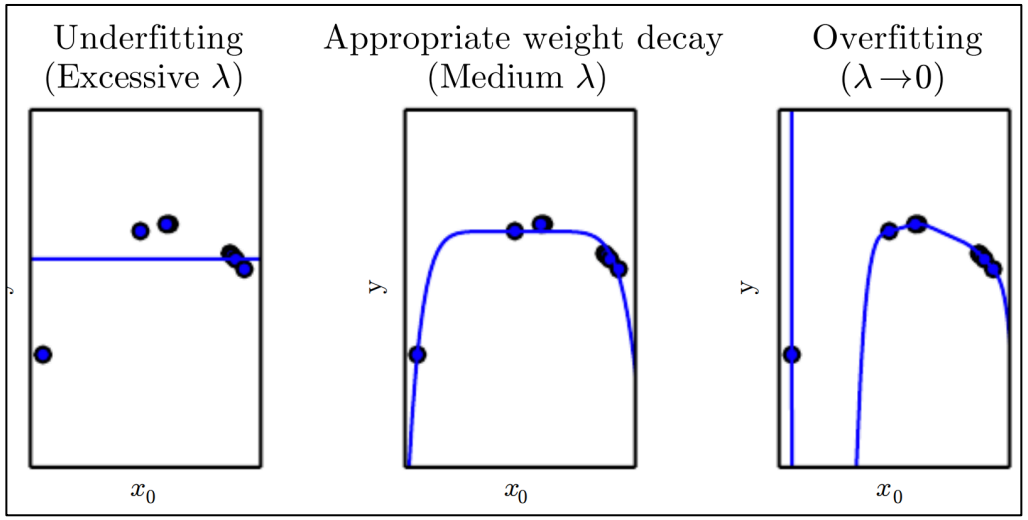
Weight Decay

- **Weight Decay** (aka L_2 **Regularization**) is a regularization technique applied to the weights of a neural network

  We add a penalizing factor to the loss function (a penalty on the L_2 Norm of the weights):

$$L_{new}(y,t) = L_{orig}(y,t) + \boldsymbol{\lambda}\|\boldsymbol{w}\|^2$$

$\boldsymbol{\lambda}$ is a value determining the strength of the penalty (encouraging smaller weights)



Dropout

- To reduce overfitting, we can apply **Dropout** to the network

  A form of regularization

- The idea is that we let a layer to randomly drop out a few output units from the layer itself during the training process

  By setting the activation to zero

  We can pick the percentage of the output units that are dropped out randomly from the applied layer. Example:

```
layers.MaxPooling2D(),
layers.Dropout(0.2),
layers.Flatten(),
```

Architecture examples

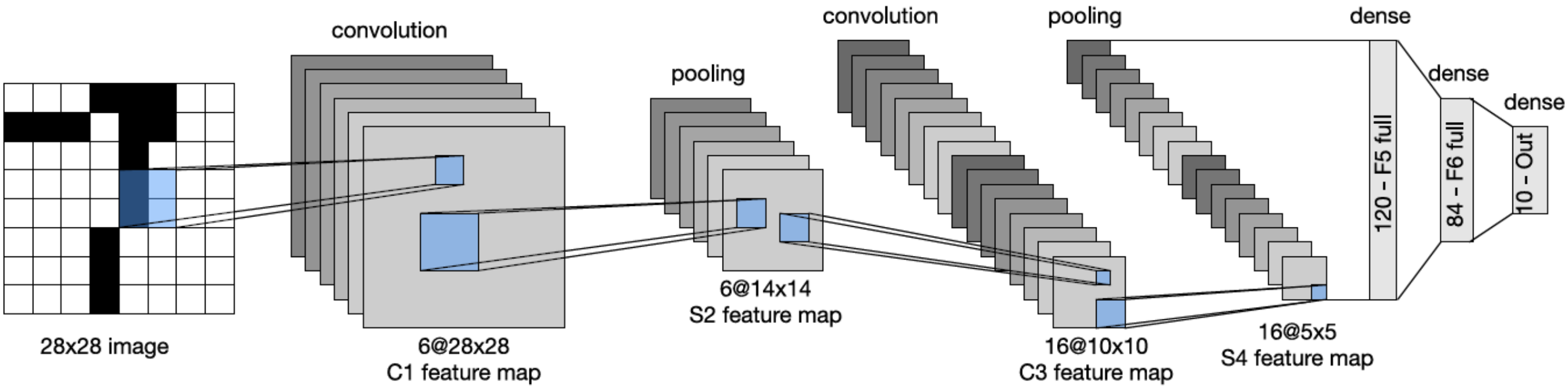
- LeNet (1998)
- AlexNet (2012)
- GoogLeNet (2014)
- ResNet (2015)
- DenseNet (2016)

LeNet

- **LeNet** is one of the earliest CNNs (1998), named after its creator Yann LeCun.

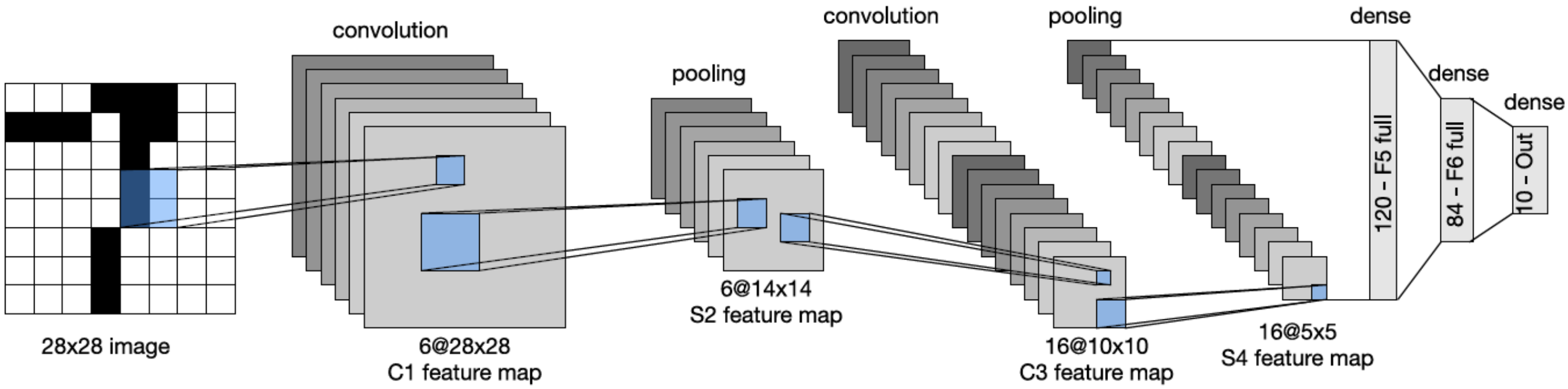
  It was designed to classify images of handwritten digits, and was trained on the MNIST dataset

  - With this CNN, after just 1 epoch, the test accuracy is already 98.8%



LeNet

- It uses tanh, weight decay, and alternates convolution layers and pooling layers



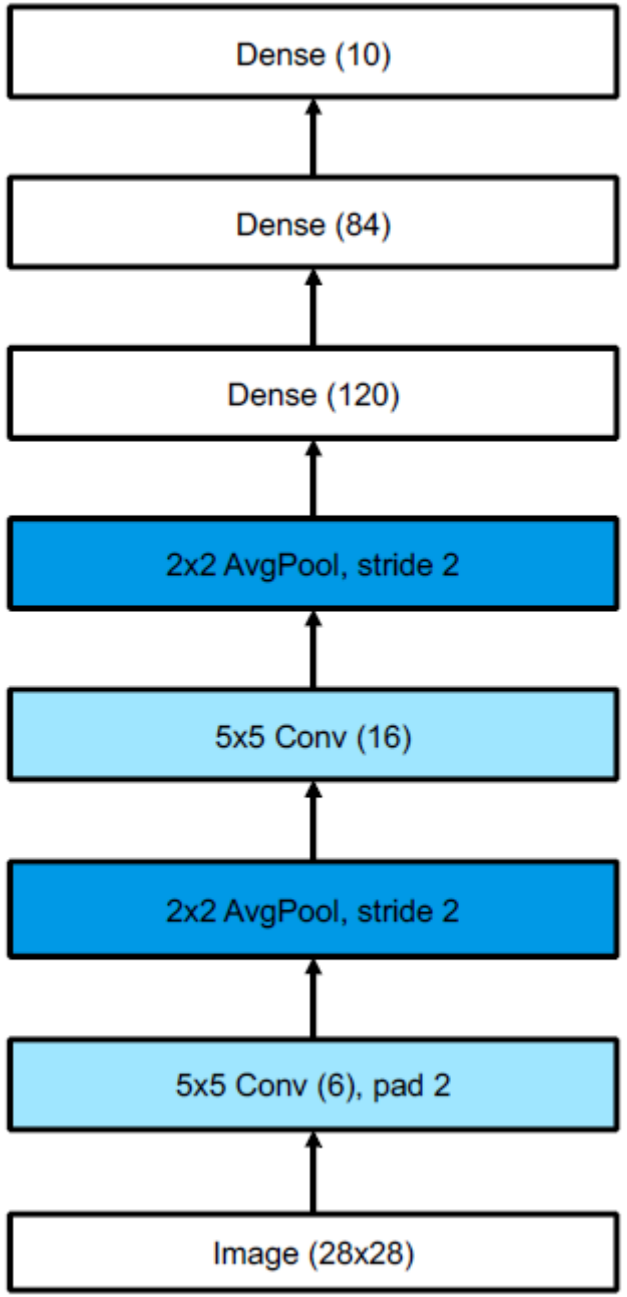
LeNet

- Image is 28x28 (MNIST)

$$(x_h + 2p_h - f_h + 1) * (x_w + 2p_w - f_w + 1)$$

$p = 2$
$f = 5$
$x = 28$

$$(28 + 4 - 5 + 1) * (28 + 4 - 5 + 1) = 28 * 28$$



LeNet

- Image is 28x28 (MNIST)

$$\frac{(x_h + 2p_h - f_h + s_h)}{s_h} * \frac{x_w + 2p_w - f_w + s_w}{s_w}$$

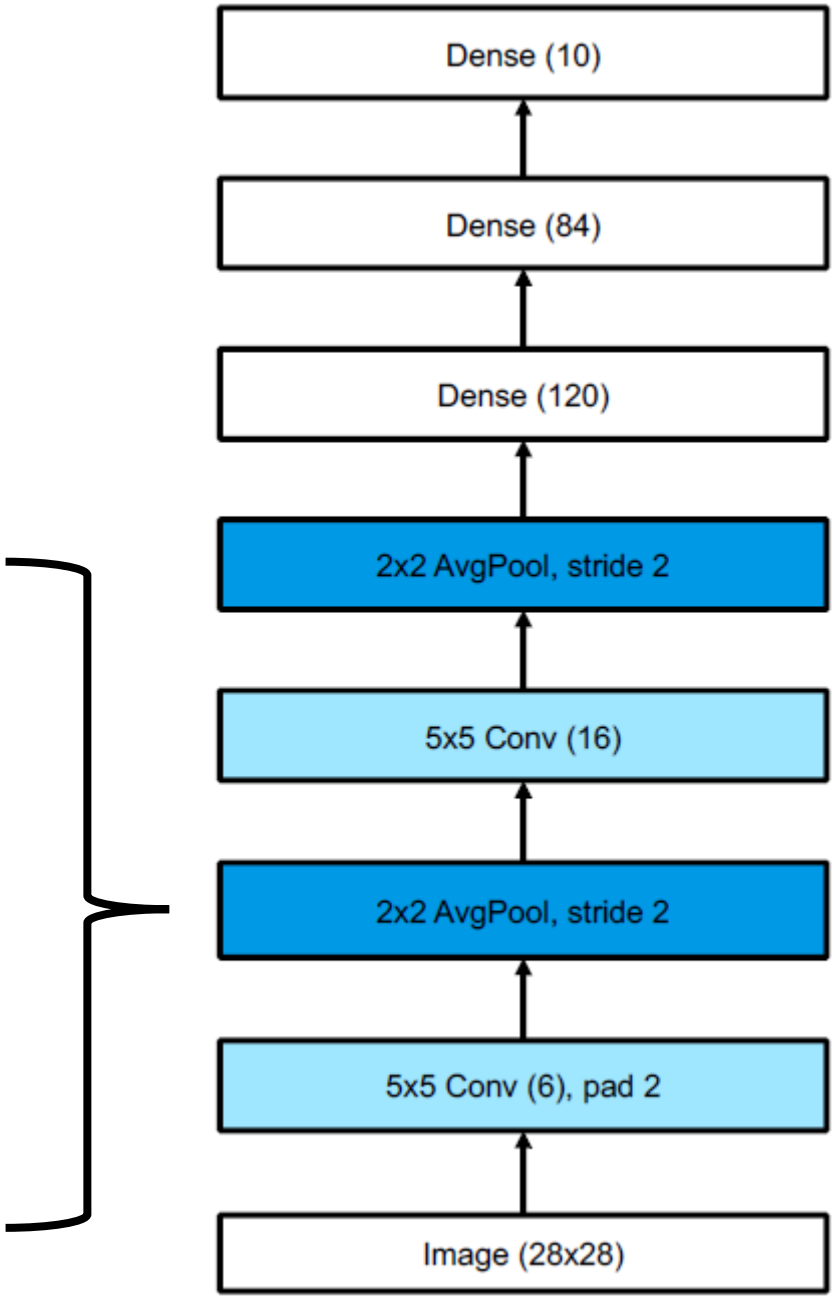
$p = 0$

$f = 2$

$x = 28$

$s = 2$

$$\frac{(28 + 0 - 2 + 2)}{2} * \frac{(28 + 0 - 2 + 2)}{2} = 14 * 14$$

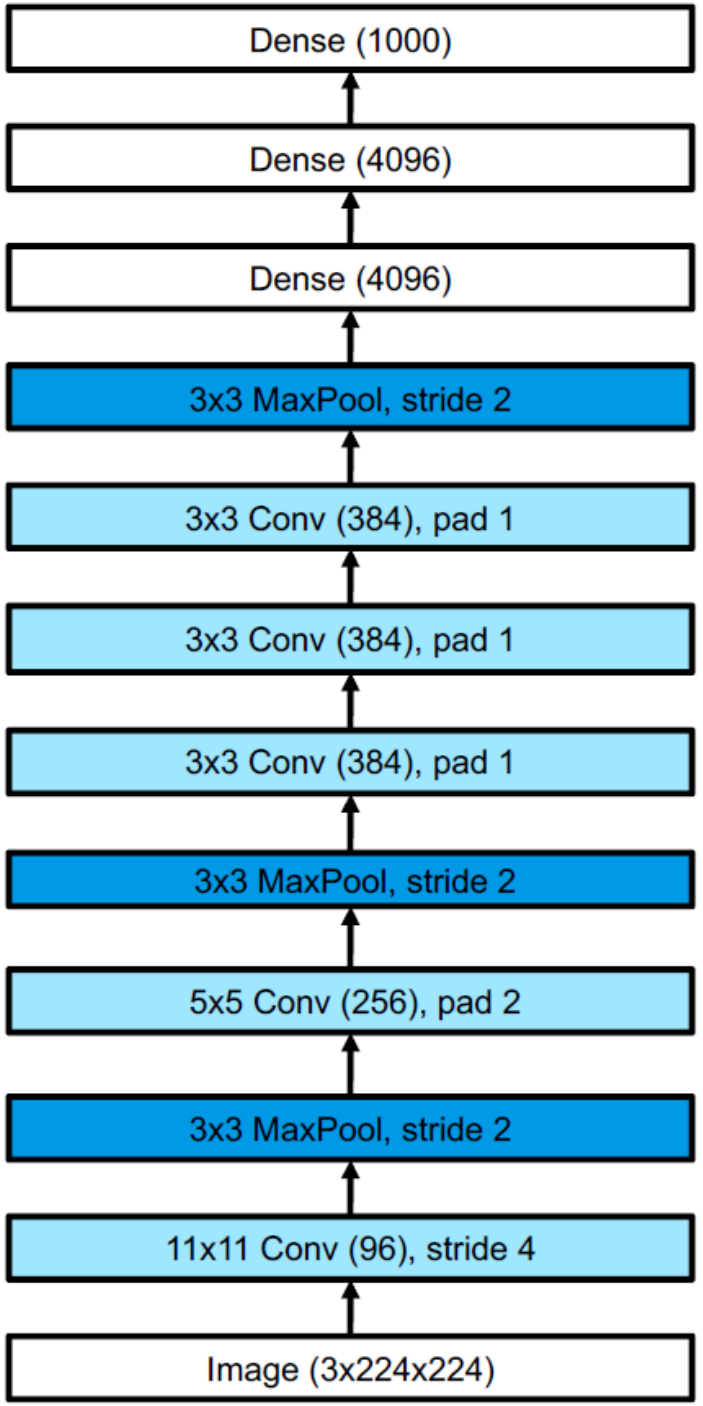


AlexNet

- AlexNet model is named after its creator Alex Krizhevsky

Very similar to LeNet, with the following differences:

- Is deeper (8 layers of adjustable parameters (that is, excluding the pooling layers) instead of 5)
- Uses ReLU instead of tanh
- Uses dropout for regularization instead of weight decay
- Stacks several convolutional layers on top of each other, instead of alternating between convolution and pooling

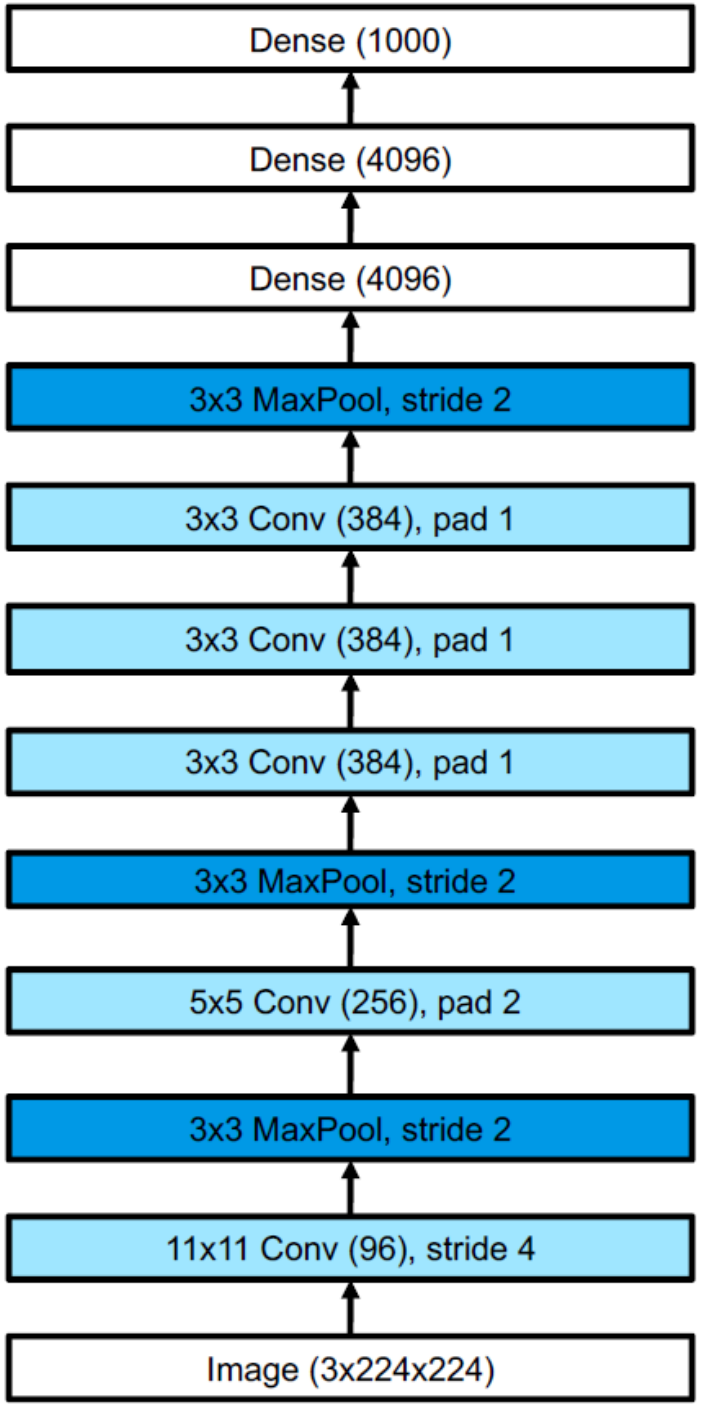


AlexNet

- When stacking multiple convolutional layers together the receptive fields become larger
  Since the output of one layer is fed into another
  - For example, three $3 * 3$ filters in a row will have a receptive field size of $7 * 7$

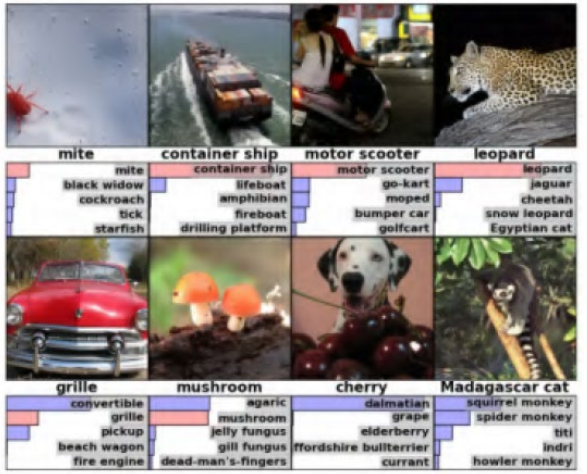
- This is better than using a single layer with a larger receptive field,
  - Because the multiple layers also have nonlinearities in between that helps

- Also, three $3 * 3$ filters have fewer parameters than one $7 * 7$



AlexNet

- In their paper, the authors showed how to reduce the (top 5) error rate on the **ImageNet** challenge from the previous best of 26% to 15%!



- **ImageNet** is a dataset of 14M images of size $256 * 256 * 3$ split in 20,000 classes

  It was used as the basis of the ImageNet Large Scale Visual Recognition Challenge (ILSVRC), which ran from 2010 to 2018 (it used a subset of 1.3M images from 1000 classes)

  ➢ Note that CNNs are not better at vision than humans

  Simply, the dataset makes many fine-grained classification distinctions (“*tiger*” / “*tiger cat*”) complex for a human

GooLeNet (Inception)

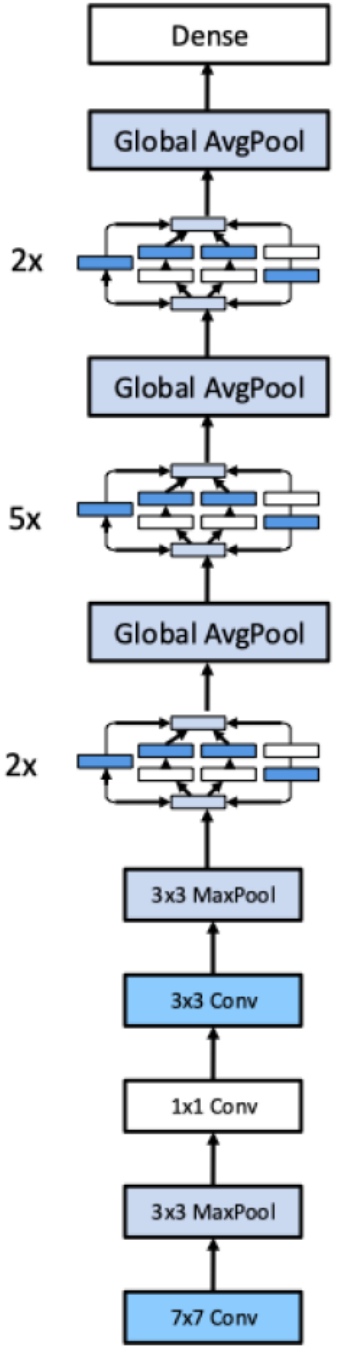
GoogLeNet is a pun on Google and LeNet

- GoogLeNet used a new kind of block, known as an **inception block**

  It employs multiple parallel pathways, each with a convolutional filter of a different size

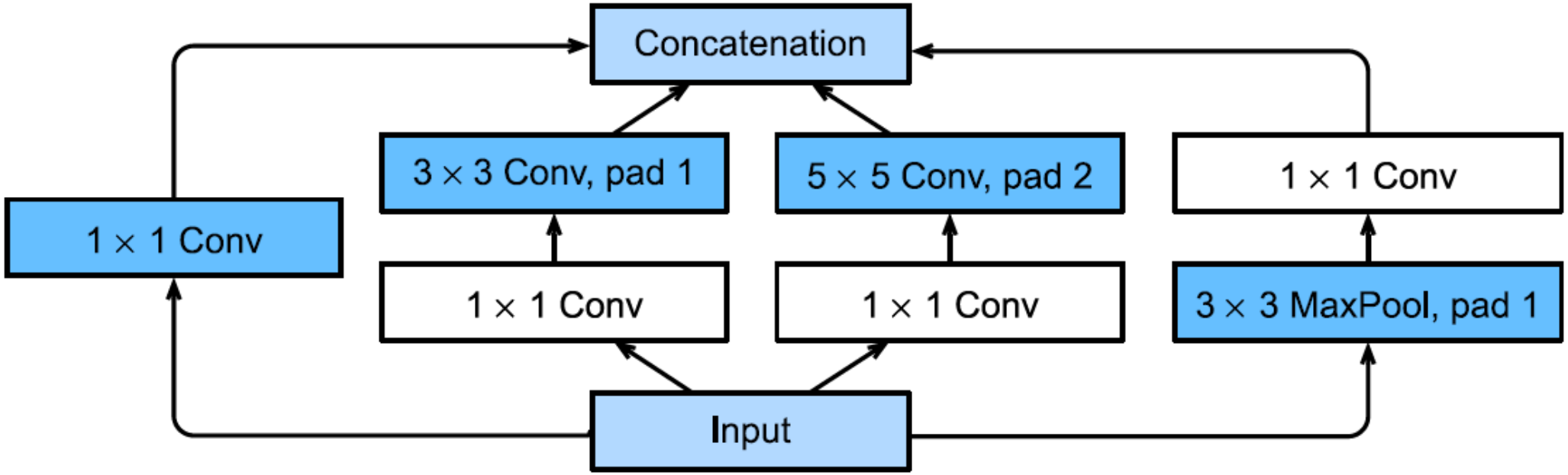
- This lets the model learn the optimal filter size at each level

  The overall model consists of 9 inception blocks followed by global average pooling



Inception module

- The $1 * 1$ convolutional layers reduce the number of channels keeping the spatial dimensions the same
- The parallel pathways allows the model to learn which filter size to use for each layer
- The concatenation block combines the outputs of all the pathways (which all have the same spatial size)



ResNet

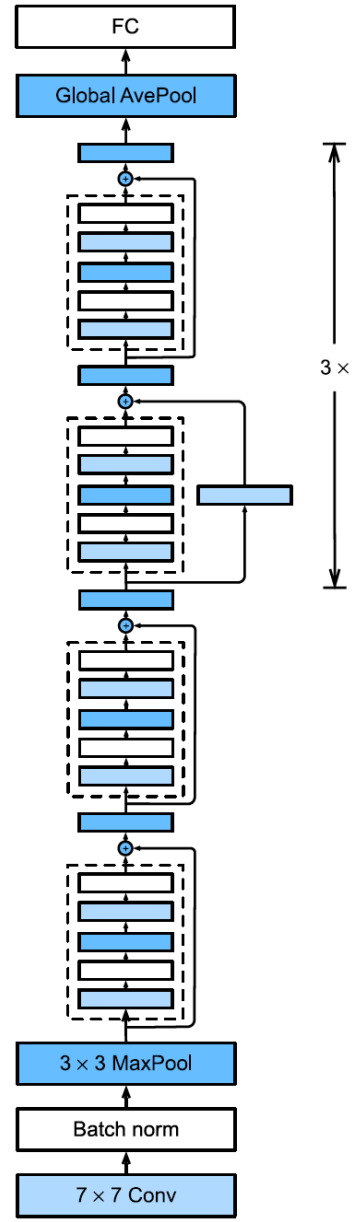
The winner of the 2015 ImageNet classification challenge was a team at Microsoft who proposed a model known as **ResNet**

- The key idea is to replace $x_{l+1} = F_l(x_l)$ with:

$$x_{l+1} = \varphi(x_l + F_l(x_l))$$

- This is known as a **residual block**, since F_l only needs to learn the residual, or difference, between input and output of this layer, which is a simpler task

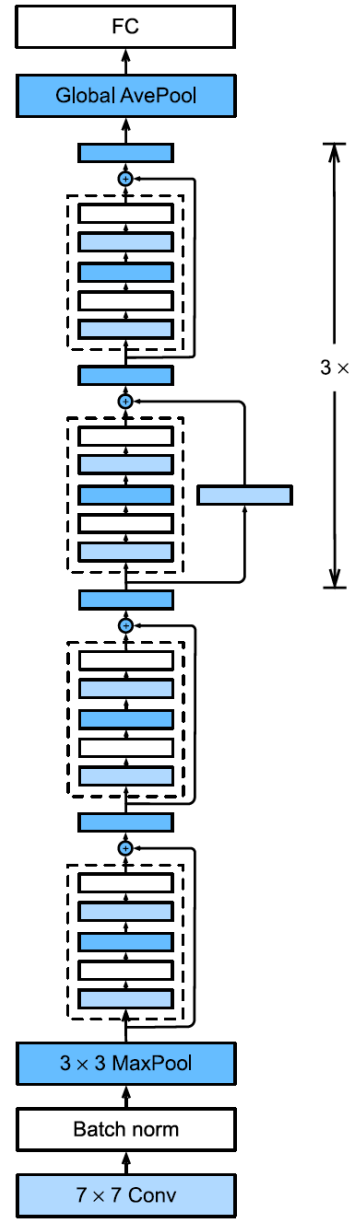
- The use of residual blocks allows us to train very deep models
  The reason this is possible is that gradient can flow directly from the output to earlier layers (residual connections)



ResNet

The **ResNet-18** architecture has 18 trainable layers:

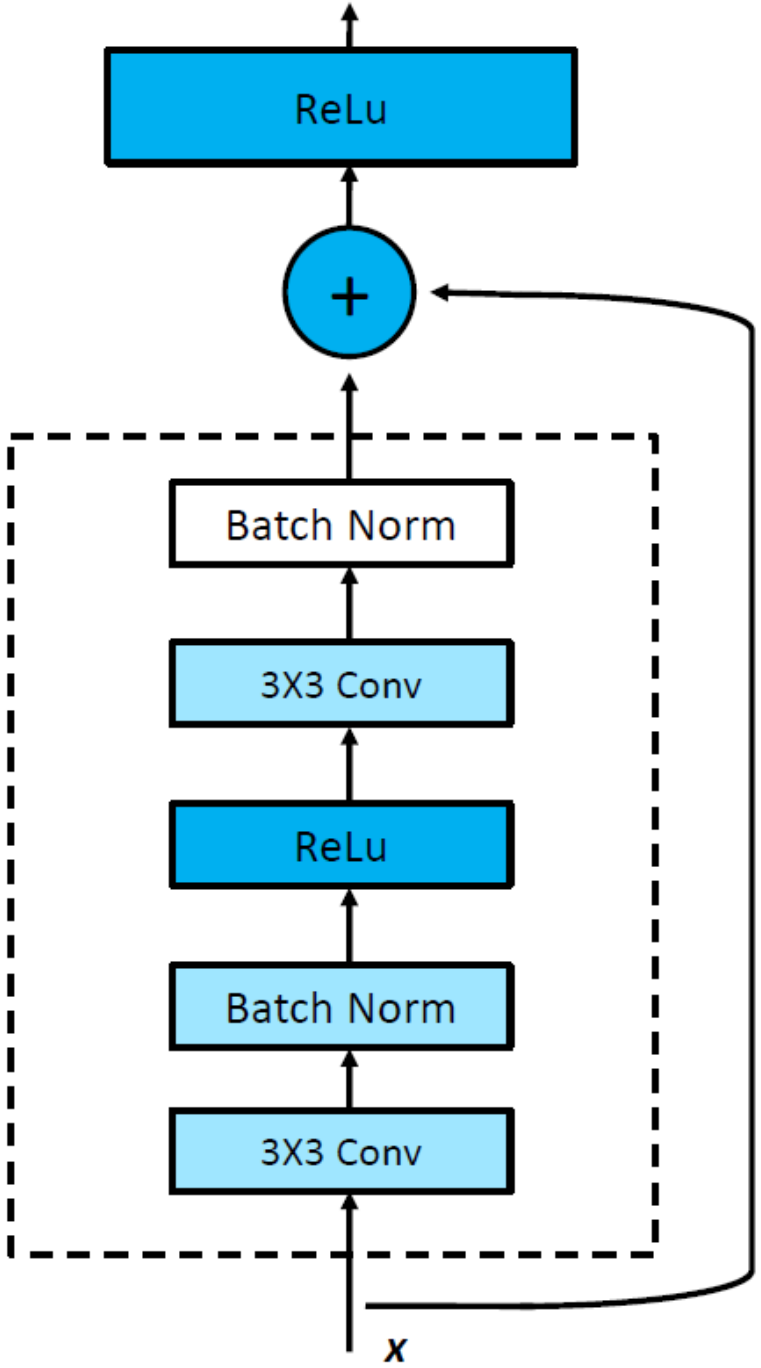
- There are 8 residual blocks with two $3 * 3$ convolutional layers in each of them
- Plus, an initial $7 * 7$ convolution (stride 2) and a final fully connected layer

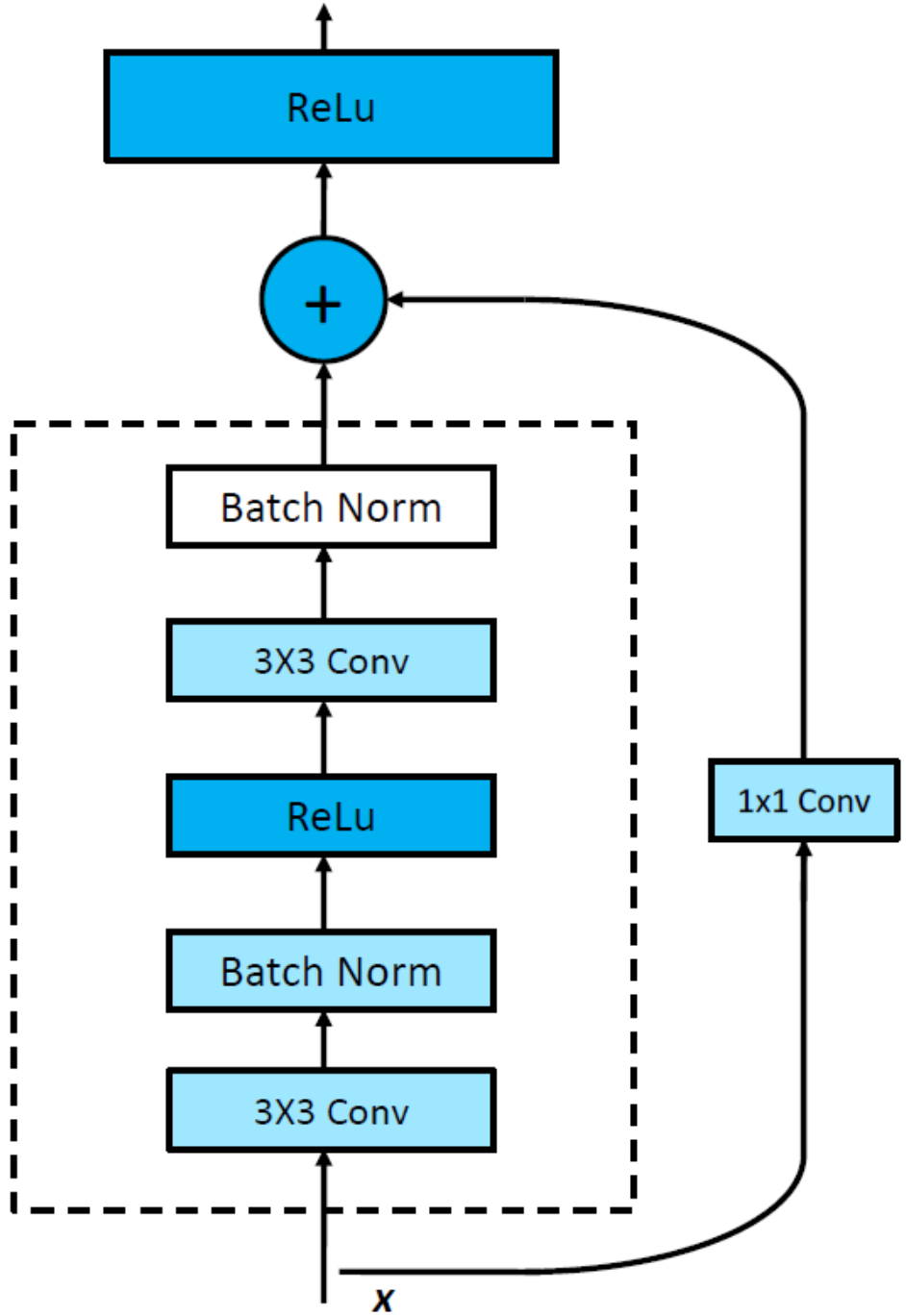


Two types of residual block, with and without $1 * 1$ conv

This also changes the number of channels

Note that we add the output of each function to its input

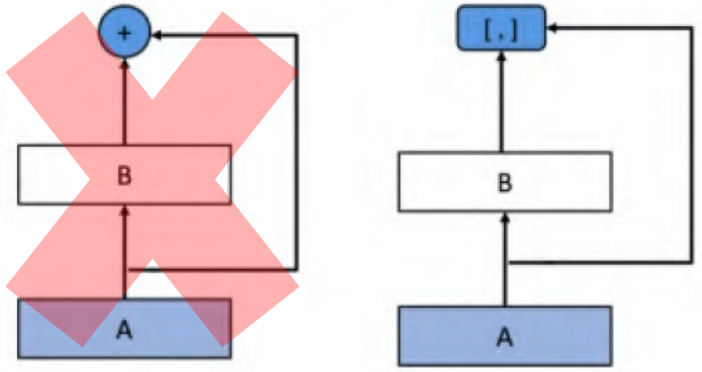


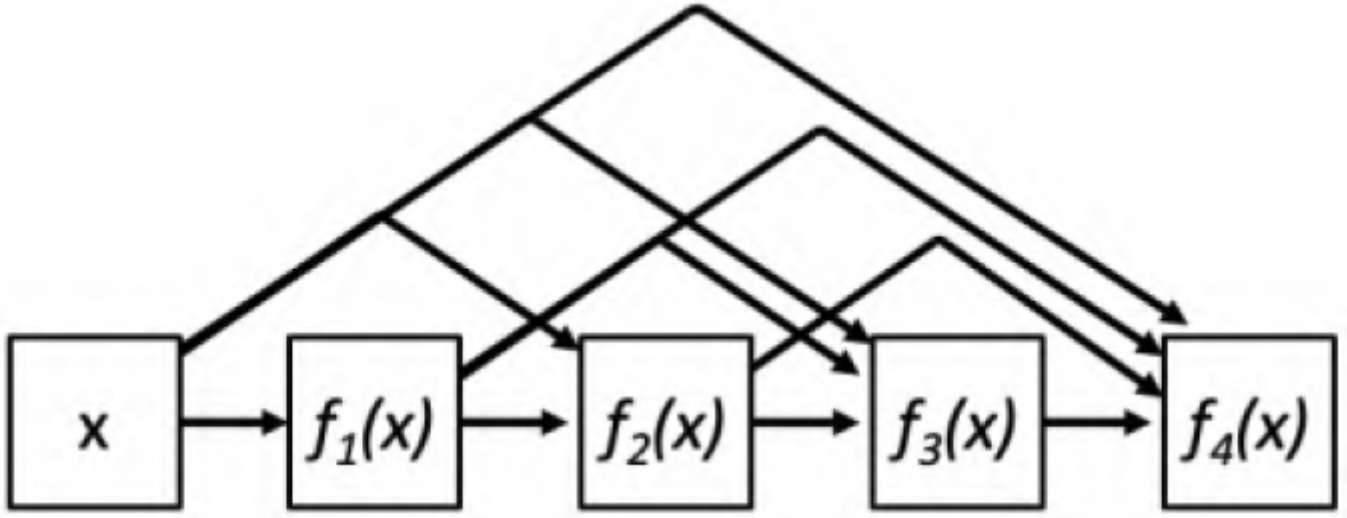


DenseNet

- Instead of adding them, we can concatenate outputs and inputs
- This is known as a **DenseNet**

  Since each layer densely depends on all previous layers



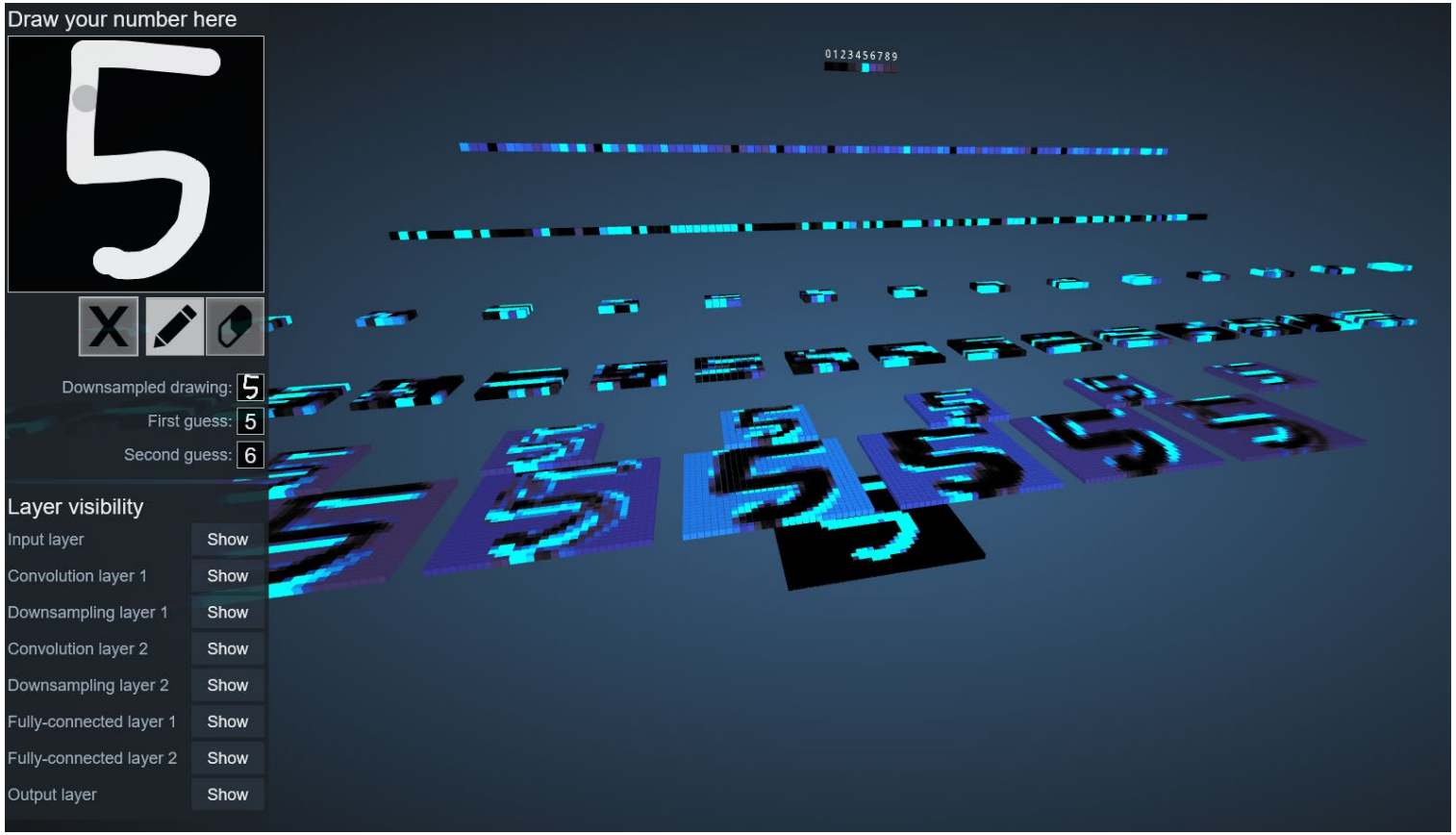


DenseNet

- The dense connectivity increases the number of parameters, since the channels get stacked depth wise
  - We can compensate for this by adding $1 * 1$ convolution layers in between
  - We can also add pooling layers with a stride of 2 to reduce the spatial resolution
- DenseNets can perform better than ResNets
  - Since all previously computed features are directly accessible to the output layer
- But they can be more computationally expensive

- Check this out:

https://www.cs.ryerson.ca/~aharley/vis/conv/



There is also a “flat” version:
https://www.cs.ryerson.ca/~aharley/vis/conv/flat

CNNs in TensorFlow

- https://www.tensorflow.org/tutorials/images/cnn

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```